

République Algérienne Démocratique et Populaire  
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

**Higher Institute of Sciences**



**Projet de fin d'étude**

**En :** Mathématiques et Informatique

**Spécialité :** Ingénierie de Systèmes d'Information & Logiciels

**Intitulé :**

---

# Vectorization Automatique d'Images Matricielles par Deep Learning

---

**Encadré par :**

ABDELLAHOUM Hamza

**Présenté par :**

BITAM Amir

AITHAMOUDI Zakaria

**Membres du Jury :**

????

????

**Soutenu le :**

19/07/2026

Année universitaire : 2025/2026

# Remerciements

Avant toute chose, nous adressons nos louanges et notre gratitude à Allah, qui nous a accordé la réussite et a mis sur notre chemin tout ce dont nous avons besoin pour atteindre ce jour tant espéré. Rien de cela n'aurait été possible sans Sa volonté.

Nous exprimons notre profonde reconnaissance à?????, présidente du jury, ainsi qu'à?????, membre du jury, pour avoir consacré de leur temps et de leur énergie afin d'évaluer notre travail avec bienveillance et rigueur.

Nos remerciements les plus sincères vont également à notre encadrant, Monsieur ABDELLAHOUM Hamza, pour sa précieuse guidance, son soutien constant et ses précieux conseils tout au long de ce projet. Ses connaissances et sa disponibilité ont grandement contribué à notre apprentissage et à l'aboutissement de ce travail.

Nous tenons aussi à remercier chaleureusement nos familles, nos amis, ainsi que toutes les personnes qui ont, de près ou de loin, contribué à la réussite de ce travail.

Enfin, nos pensées et nos prières accompagnent nos frères et sœurs en Palestine, en espérant la paix, la justice et la fin de leur souffrance.

# Résumé

La vectorisation automatique d’images consiste à convertir une représentation matricielle, composée de pixels, en une représentation vectorielle décrite par des primitives géométriques. Cette transformation permet notamment d’obtenir des graphiques redimensionnables et éditables, mais elle reste difficile pour les images naturelles, dont les couleurs, les contours et les textures doivent être approximés par un ensemble limité de formes. Dans ce mémoire, nous étudions la génération automatique de fichiers SVG à partir d’images matricielles couleur au moyen de l’apprentissage profond.

L’approche proposée décompose d’abord l’image en superpixels avec l’algorithme SLIC. Chaque région est ensuite recadrée et encodée par un Vision Transformer DINOv3 pré-entraîné. Des requêtes apprenables, associées aux chemins à produire, exploitent les caractéristiques visuelles au moyen de mécanismes de cross-attention et de self-attention. Le réseau prédit ainsi des chemins fermés constitués de quatre courbes de Bézier cubiques et d’une couleur de remplissage. L’entraînement ne nécessite aucun SVG de référence : DiffVG rasterise les primitives prédites afin de calculer une perte de reconstruction normalisée et une pénalité de débordement hors du superpixel. À l’inférence, les chemins locaux sont replacés dans le repère de l’image d’origine, puis assemblés dans un document SVG.

Les expérimentations confirment la production de fichiers SVG valides et rechargeables. Sur le jeu interne de douze images, la configuration à 128 chemins par superpixel obtient une MSE de 0,00573, un PSNR de 24,13 dB, une SSIM de 0,645 et une LPIPS de 0,290. Cependant, les budgets d’entraînement des configurations à 32 et 128 chemins diffèrent, ce qui ne permet pas d’isoler l’effet du seul nombre de primitives. Sur le benchmark externe de quatre images, SuperSVG conserve la meilleure fidélité, tandis que notre approche atteint un temps moyen de 0,684 seconde, contre 2,548 secondes pour SuperSVG et 243,804 secondes pour LIVE, au prix de SVG plus complexes. Ces résultats mettent en évidence la faisabilité de la prédiction directe de primitives et le compromis entre fidélité, rapidité et éditabilité.

**Mots-clés :** Vectorisation d’images, SVG, DINOv3, Vision Transformer, superpixels SLIC, rendu différentiable, courbes de Bézier.

# Abstract

Automatic image vectorization converts a raster representation made of pixels into a vector representation described by geometric primitives. This transformation produces scalable and editable graphics, but remains difficult for natural images, whose colors, boundaries, and textures must be approximated by a limited set of shapes. In this thesis, we investigate the automatic generation of SVG files from color raster images using deep learning.

The proposed approach first decomposes the input image into superpixels using SLIC. Each region is then cropped and encoded by a pretrained DINOv3 Vision Transformer. Learnable queries associated with the paths to be generated exploit the visual features through cross-attention and self-attention mechanisms. The network predicts closed paths composed of four cubic Bézier curves and a fill color. Training does not require reference SVG files : DiffVG rasterizes the predicted primitives to compute a normalized reconstruction loss and a penalty for shapes extending beyond the superpixel. During inference, local paths are mapped back to the coordinate system of the original image and assembled into an SVG document.

The experiments confirm that the system produces valid and reloadable SVG files. On the internal set of twelve images, the configuration with 128 paths per superpixel achieves an MSE of 0.00573, a PSNR of 24.13 dB, an SSIM of 0.645, and an LPIPS of 0.290 ; however, the different training budgets of the 32-path and 128-path configurations prevent us from isolating the effect of the number of primitives alone. On the external four-image benchmark, SuperSVG retains the highest fidelity, whereas our approach reaches an average processing time of 0.684 seconds, compared with 2.548 seconds for SuperSVG and 243.804 seconds for LIVE, at the cost of more complex SVG documents. These results highlight the feasibility of direct primitive prediction and the trade-off between fidelity, speed, and editability.

**Keywords :** Image vectorization, SVG, DINOv3, Vision Transformer, SLIC superpixels, differentiable rendering, Bézier curves.

# Liste des Acronymes

|               |                                           |
|---------------|-------------------------------------------|
| <b>BMP</b>    | Bitmap Image File                         |
| <b>CNN</b>    | Convolutional Neural Network              |
| <b>CUDA</b>   | Compute Unified Device Architecture       |
| <b>DINO</b>   | Self-Distillation with No Labels          |
| <b>DiffVG</b> | Differentiable Vector Graphics            |
| <b>FFN</b>    | Feed-Forward Network                      |
| <b>GELU</b>   | Gaussian Error Linear Unit                |
| <b>GPU</b>    | Graphics Processing Unit                  |
| <b>JPEG</b>   | Joint Photographic Experts Group          |
| <b>JSON</b>   | JavaScript Object Notation                |
| <b>LIVE</b>   | Layer-wise Image Vectorization            |
| <b>LN</b>     | Layer Normalization                       |
| <b>LPIPS</b>  | Learned Perceptual Image Patch Similarity |
| <b>LSTM</b>   | Long Short-Term Memory                    |
| <b>MHA</b>    | Multi-Head Attention                      |
| <b>MSE</b>    | Mean Squared Error                        |
| <b>PNG</b>    | Portable Network Graphics                 |
| <b>PSNR</b>   | Peak Signal-to-Noise Ratio                |
| <b>RGB</b>    | Red, Green and Blue                       |
| <b>RGBA</b>   | Red, Green, Blue and Alpha                |
| <b>RNN</b>    | Recurrent Neural Network                  |
| <b>SLIC</b>   | Simple Linear Iterative Clustering        |
| <b>SSIM</b>   | Structural Similarity Index Measure       |
| <b>SVG</b>    | Scalable Vector Graphics                  |
| <b>TIFF</b>   | Tagged Image File Format                  |
| <b>UDF</b>    | Unsigned Distance Field                   |
| <b>ViT</b>    | Vision Transformer                        |
| <b>XML</b>    | Extensible Markup Language                |

# Table des matières

|                                                                               |          |
|-------------------------------------------------------------------------------|----------|
| Remerciements                                                                 | i        |
| Résumé                                                                        | ii       |
| Abstract                                                                      | iii      |
| Liste des Acronymes                                                           | iv       |
| Introduction Générale                                                         | 2        |
| <b>1 État de l’Art</b>                                                        | <b>3</b> |
| 1.1 Introduction . . . . .                                                    | 3        |
| 1.2 La vectorisation d’images . . . . .                                       | 3        |
| 1.2.1 Images matricielles . . . . .                                           | 3        |
| A. Pixels . . . . .                                                           | 3        |
| 1.2.2 Images vectorielles . . . . .                                           | 4        |
| A. Courbes de Bézier . . . . .                                                | 5        |
| B. Format SVG . . . . .                                                       | 5        |
| 1.3 Apprentissage automatique et apprentissage profond . . . . .              | 5        |
| 1.3.1 Apprentissage automatique et apprentissage auto-supervisé . . .         | 6        |
| 1.3.2 Apprentissage profond . . . . .                                         | 6        |
| A. Réseaux de neurones . . . . .                                              | 7        |
| B. Transformers . . . . .                                                     | 7        |
| C. Vision Transformers . . . . .                                              | 8        |
| 1.4 Mécanismes d’attention . . . . .                                          | 9        |
| 1.4.1 Self-attention . . . . .                                                | 10       |
| 1.4.2 Cross-attention . . . . .                                               | 11       |
| 1.5 Concepts utilisés dans la vectorisation par apprentissage profond . . . . | 11       |
| 1.5.1 Superpixels . . . . .                                                   | 12       |
| 1.5.2 Rendu différentiable . . . . .                                          | 12       |
| 1.5.3 Fonctions de perte . . . . .                                            | 12       |
| 1.5.4 Apprentissage coarse-to-fine . . . . .                                  | 13       |
| 1.6 Évaluation des résultats . . . . .                                        | 13       |
| 1.6.1 MSE . . . . .                                                           | 13       |
| 1.6.2 PSNR . . . . .                                                          | 14       |
| 1.6.3 SSIM . . . . .                                                          | 14       |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 1.6.4    | LPIPS . . . . .                                        | 14        |
| 1.6.5    | Temps d'inférence . . . . .                            | 15        |
| 1.7      | Travaux connexes . . . . .                             | 15        |
| 1.7.1    | Approches algorithmiques . . . . .                     | 15        |
| A.       | Potrace . . . . .                                      | 15        |
| 1.7.2    | Approches basées sur l'optimisation . . . . .          | 16        |
| A.       | DiffVG . . . . .                                       | 16        |
| B.       | LIVE . . . . .                                         | 16        |
| 1.7.3    | Approches basées sur l'apprentissage profond . . . . . | 16        |
| A.       | Im2Vec . . . . .                                       | 17        |
| B.       | SuperSVG . . . . .                                     | 17        |
| 1.8      | Synthèse comparative des approches . . . . .           | 18        |
| 1.9      | Conclusion . . . . .                                   | 19        |
| <b>2</b> | <b>Approche proposée</b>                               | <b>20</b> |
| 2.1      | Introduction . . . . .                                 | 20        |
| 2.2      | Vue d'ensemble de la méthode . . . . .                 | 20        |
| 2.2.1    | Principe général . . . . .                             | 21        |
| 2.2.2    | Caractéristiques de l'approche proposée . . . . .      | 21        |
| 2.3      | Préparation des données . . . . .                      | 22        |
| 2.3.1    | Jeu de données utilisé . . . . .                       | 23        |
| 2.3.2    | Décomposition en superpixels . . . . .                 | 23        |
| 2.3.3    | Recadrage et construction de l'entrée . . . . .        | 23        |
| 2.4      | Architecture du modèle . . . . .                       | 24        |
| 2.4.1    | Extraction des caractéristiques avec DINOv3 . . . . .  | 24        |
| 2.4.2    | Requêtes apprenables et cross-attention . . . . .      | 25        |
| 2.4.3    | Interactions entre les chemins . . . . .               | 26        |
| 2.4.4    | Représentation d'un chemin vectoriel . . . . .         | 26        |
| 2.5      | Rendu différentiable et fonctions de perte . . . . .   | 27        |
| 2.5.1    | Construction de la scène DiffVG . . . . .              | 28        |
| 2.5.2    | Perte de reconstruction normalisée . . . . .           | 28        |
| 2.5.3    | Pénalité de débordement . . . . .                      | 29        |
| 2.5.4    | Fonction objective . . . . .                           | 29        |
| 2.6      | Stratégie d'entraînement . . . . .                     | 30        |
| 2.6.1    | Optimisation des paramètres . . . . .                  | 30        |
| 2.6.2    | Sauvegarde et reprise . . . . .                        | 32        |
| 2.7      | Inférence et génération du SVG . . . . .               | 32        |
| 2.7.1    | Traitement de l'image complète . . . . .               | 32        |
| 2.7.2    | Algorithme d'inférence . . . . .                       | 33        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 2.7.3    | Différences entre entraînement et inférence . . . . . | 34        |
| 2.7.4    | Périmètre de la version implémentée . . . . .         | 34        |
| 2.8      | Conclusion . . . . .                                  | 35        |
| <b>3</b> | <b>Expérimentations et Résultats</b>                  | <b>36</b> |
| 3.1      | Introduction . . . . .                                | 36        |
| 3.2      | Environnement expérimental . . . . .                  | 36        |
| 3.2.1    | Configuration matérielle et logicielle . . . . .      | 36        |
| 3.2.2    | Configurations de notre approche . . . . .            | 37        |
| 3.3      | Benchmark et conditions d'évaluation . . . . .        | 38        |
| 3.3.1    | Construction du jeu de test . . . . .                 | 38        |
| 3.3.2    | Conditions d'inférence . . . . .                      | 39        |
| 3.3.3    | Métriques et complexité vectorielle . . . . .         | 40        |
| 3.4      | Validation technique de l'implémentation . . . . .    | 40        |
| 3.4.1    | Test du rendu différentiable . . . . .                | 40        |
| 3.4.2    | Validation des checkpoints et des SVG . . . . .       | 40        |
| 3.5      | Évolution de l'apprentissage . . . . .                | 41        |
| 3.6      | Comparaison avec les approches de référence . . . . . | 42        |
| 3.6.1    | Configurations retenues . . . . .                     | 43        |
| 3.6.2    | Résultats quantitatifs communs . . . . .              | 43        |
| 3.6.3    | Temps et complexité vectorielle . . . . .             | 45        |
| 3.6.4    | Comparaison qualitative . . . . .                     | 46        |
| 3.6.5    | Portée de la comparaison . . . . .                    | 48        |
| 3.7      | Étude interne des configurations proposées . . . . .  | 48        |
| 3.7.1    | Fidélité de reconstruction . . . . .                  | 48        |
| 3.7.2    | Temps d'inférence et complexité du SVG . . . . .      | 49        |
| 3.8      | Analyse qualitative de l'approche proposée . . . . .  | 51        |
| 3.9      | Discussion et limites . . . . .                       | 52        |
| 3.10     | Conclusion . . . . .                                  | 53        |
|          | <b>Conclusion Générale</b>                            | <b>54</b> |
|          | <b>Bibliographie</b>                                  | <b>55</b> |

# Table des figures

|     |                                                                                                                                |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Comparaison entre une image matricielle (a) et une image vectorielle (b).                                                      | 4  |
| 1.2 | Exemple du contenu d'un fichier .svg . . . . .                                                                                 | 5  |
| 1.3 | Structure simplifiée d'un encodeur Transformer . . . . .                                                                       | 8  |
| 1.4 | Architecture générale d'un Vision Transformer . . . . .                                                                        | 8  |
| 1.5 | Attention par produit scalaire mise à l'échelle . . . . .                                                                      | 10 |
| 2.1 | Pipeline générale de l'approche proposée, de l'image matricielle aux chemins SVG. . . . .                                      | 21 |
| 2.2 | Chemin fermé composé de quatre courbes de Bézier cubiques et de douze points prédits. . . . .                                  | 27 |
| 3.1 | Exemple de segmentation SLIC appliquée à une image du jeu de test. .                                                           | 39 |
| 3.2 | Évolution des métriques sur le jeu de test pour la configuration à 32 chemins. . . . .                                         | 42 |
| 3.3 | Comparaison des quatre métriques de fidélité; les barres d'erreur représentent l'écart-type entre les images. . . . .          | 44 |
| 3.4 | Compromis entre fidélité, temps de vectorisation et complexité des trois approches retenues pour l'analyse temporelle. . . . . | 46 |
| 3.5 | Comparaison qualitative des SVG rasterisés par la procédure commune.                                                           | 47 |
| 3.6 | Comparaison quantitative des configurations finales; les barres d'erreur représentent l'écart-type. . . . .                    | 49 |
| 3.7 | Répartition du temps moyen et évolution de la complexité vectorielle. .                                                        | 50 |
| 3.8 | Comparaison qualitative sur une scène complexe, un objet isolé et une image fortement texturée. . . . .                        | 51 |

# Liste des tableaux

|     |                                                                                                  |    |
|-----|--------------------------------------------------------------------------------------------------|----|
| 1.1 | Synthèse comparative des principales approches de vectorisation étudiées.                        | 18 |
| 2.1 | Principaux modules de la pipeline proposée. . . . .                                              | 22 |
| 2.2 | Configuration enregistrée pour l'entraînement du modèle à 128 chemins<br>par superpixel. . . . . | 31 |
| 2.3 | Comparaison du flux d'entraînement et du flux d'inférence. . . . .                               | 34 |
| 3.1 | Environnement matériel et logiciel utilisé pour les expérimentations. . .                        | 37 |
| 3.2 | Configurations finales comparées pendant l'évaluation. . . . .                                   | 38 |
| 3.3 | Résultats des tests techniques de l'implémentation. . . . .                                      | 41 |
| 3.4 | Évolution des métriques moyennes du modèle à 32 chemins. . . . .                                 | 41 |
| 3.5 | Configurations utilisées pour la comparaison entre approches. . . . .                            | 43 |
| 3.6 | Comparaison de la fidélité des approches sur les quatre images communes.                         | 44 |
| 3.7 | Temps et complexité des SVG produits sur les quatre images communes.                             | 45 |
| 3.8 | Résultats moyens de fidélité sur les 12 images du jeu de test. . . . .                           | 48 |
| 3.9 | Coût moyen de l'inférence et complexité des SVG générés. . . . .                                 | 50 |

# Introduction Générale

Les images numériques sont principalement représentées sous forme matricielle ou vectorielle. Une image matricielle décrit son contenu par une grille de pixels et convient particulièrement aux photographies, mais sa qualité dépend de la résolution. Une image vectorielle repose au contraire sur des primitives géométriques redimensionnables et éditables [1, 2]. Le passage automatique d’une représentation à l’autre reste difficile, car il faut retrouver les régions, les contours et les couleurs de l’image tout en limitant la complexité du document produit.

Dans ce contexte, notre projet vise à générer automatiquement un fichier SVG à partir d’une image matricielle couleur. L’image est d’abord décomposée en superpixels avec SLIC [3]. Chaque région est ensuite encodée par un backbone DINOv3, puis un décodeur fondé sur la cross-attention et la self-attention prédit des chemins fermés composés de courbes de Bézier cubiques et d’une couleur de remplissage. DiffVG permet d’entraîner l’ensemble à partir d’une supervision matricielle, sans disposer de SVG de référence [4, 5].

L’évaluation associe des métriques de fidélité à des mesures de temps et de complexité des fichiers générés. Une comparaison avec SuperSVG, LIVE et Im2Vec situe notre système parmi les approches de référence, tandis qu’une étude interne examine les deux configurations entraînées au cours du projet.

Ce mémoire est organisé comme suit :

- **Chapitre 1** : il introduit les représentations matricielle et vectorielle, les architectures d’apprentissage profond, les mécanismes d’attention, le rendu différentiable et les principales approches de vectorisation existantes.
- **Chapitre 2** : il décrit la préparation des données, l’architecture fondée sur DINOv3, la représentation des chemins, les fonctions de perte, l’entraînement et la génération du document SVG final.
- **Chapitre 3** : il précise l’environnement et le benchmark, valide la chaîne de rendu, puis analyse quantitativement et qualitativement les résultats obtenus ainsi que leur comparaison avec les approches de référence.

Enfin, une conclusion générale synthétise les apports de ce travail, ses limites et les améliorations qui pourraient prolonger l’approche proposée.

# Chapitre 1

## État de l'Art

### 1.1 Introduction

Dans ce chapitre, nous exposerons d'abord les concepts clés à la vectorisation d'images, à l'apprentissage automatique, et les architectures utilisées tout au long de ce travail, ceci nous permettra de mieux appréhender ensuite les travaux effectués sur le sujet.

### 1.2 La vectorisation d'images

Dans cette section nous tenterons de définir les termes et concepts liés à la transformation d'images matricielles en images vectorielles, dite, vectorisation d'images. En soulignant notamment la différence entre les images dites matricielles et vectorielles. En opposant ainsi les deux types d'images nous pourrions mieux cerner la motivation derrière ce travail et pourquoi la vectorisation d'image constitue un exercice complexe de la vision par ordinateur.

#### 1.2.1 Images matricielles

Les images matricielles peuvent être définies comme des fonctions bidimensionnelles  $f(x, y)$ , où  $x$  et  $y$  représentent des coordonnées spatiales planes et où la valeur de  $f$  associée à chaque paire de coordonnées correspond à l'intensité ou à la couleur du point considéré. Plus simplement, elles peuvent être représentées comme une grille de pixels organisée selon une largeur et une hauteur données [1]. Les images matricielles présentent toutefois une limite lors de leur agrandissement. En effet, comme elles sont constituées d'un nombre fixe de pixels, une augmentation importante de leurs dimensions peut rendre ces derniers visibles et produire un effet de pixellisation. La qualité de l'image se dégrade alors progressivement à mesure que le facteur d'agrandissement augmente.

#### A. Pixels

Un pixel, contraction de *picture element*, représente l'unité élémentaire d'une image numérique. Il correspond à un point de la grille matricielle et possède une position, définie par ses coordonnées spatiales, ainsi qu'une valeur décrivant son intensité ou sa couleur. Dans une image en niveaux de gris, cette valeur correspond généralement à une intensité

lumineuse, tandis que dans une image couleur elle est souvent représentée par plusieurs composantes, par exemple rouge, vert et bleu. Ainsi, une image matricielle peut être comprise comme un ensemble fini de pixels organisés selon une largeur et une hauteur données [1]. Cette représentation rend le contenu directement dépendant de la résolution : lorsqu'une image est fortement agrandie, les pixels deviennent progressivement visibles, ce qui produit l'effet de pixellisation. Ainsi, le pixel constitue l'unité élémentaire de l'image matricielle, mais il impose également une granularité fixe qui limite la conservation des détails lors des changements d'échelle.

## 1.2.2 Images vectorielles

Les images vectorielles sont décrites par un ensemble de formes géométriques individuelles, telles que des cercles, des segments de droite ou des courbes de Bézier. Chacune de ces formes est enregistrée avec les paramètres mathématiques et les attributs qui la définissent, notamment ses points d'ancrage, sa couleur de remplissage ou l'épaisseur de son contour. Contrairement aux images matricielles, dont le nombre de pixels est fixe, les images vectorielles peuvent être agrandies sans perte de qualité, car leur rendu est recalculé pour chaque résolution [2]. Comme l'illustre la figure 1.1, l'image matricielle présentée en (a) devient pixellisée lors de son agrandissement, tandis que l'image vectorielle présentée en (b) conserve des contours nets grâce à sa définition mathématique. Les images vectorielles sont donc largement utilisées dans les domaines du graphisme et de la conception. Cependant, la transformation d'une image matricielle en une image vectorielle reste complexe en raison de la différence entre leurs modes de représentation.

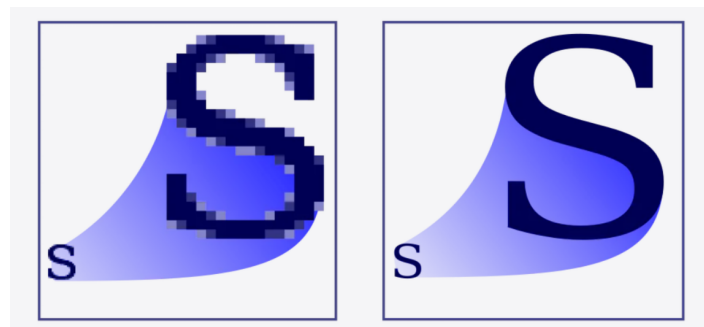


FIGURE 1.1 – Comparaison entre une image matricielle (a) et une image vectorielle (b).

Afin de mieux comprendre la manière dont une image vectorielle est construite et enregistrée, il convient de présenter deux éléments essentiels. Les courbes de Bézier permettent de représenter des contours et des formes complexes à partir d'un nombre limité de points de contrôle, tandis que le format SVG fournit une structure permettant de décrire et de stocker ces primitives vectorielles.

## A. Courbes de Bézier

Une courbe de Bézier est définie comme une fonction polynomiale dont la forme est dictée par la position de points de contrôle. La courbe passe par le premier et le dernier point (points d'ancrage), tandis que les points intermédiaires agissent comme des "aimants" qui tirent la courbe vers eux [2].

Les courbes de Bézier font partie des formes géométriques individuelles qui composent les images vectorielles et se distinguent par le faible nombre de données numériques nécessaires à la représentation de formes complexes.

## B. Format SVG

SVG (Scalable Vector Graphics) est un format basé sur XML comportant des directives pour la définition de graphiques 2D[6]. Stockées dans des fichiers textes rendant les SVG compressible et indexable, ces directives SVG supportent aussi bien les formes géométriques vectorielle que les images et le texte. Pour les raisons précédentes ce sera ce format qui sera préféré pour la sortie du modèle.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://
www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg width="391" height="391" viewBox="-70.5 -70.5 391 391"
xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://
www.w3.org/1999/xlink">
<rect fill="#fff" stroke="#000" x="-70" y="-70" width="390"
height="390"/>
<g opacity="0.8">
  <rect x="25" y="25" width="200" height="200" fill="lime"
stroke-width="4" stroke="pink" />
  <circle cx="125" cy="125" r="75" fill="orange" />
  <polyline points="50,150 50,200 200,200 200,100" stroke="red"
stroke-width="4" fill="none" />
  <line x1="50" y1="50" x2="200" y2="200" stroke="blue" stroke-
width="4" />
</g>
</svg>
```

FIGURE 1.2 – Exemple du contenu d'un fichier .svg

## 1.3 Apprentissage automatique et apprentissage profond

Cette section présente les notions d'apprentissage automatique et d'apprentissage profond nécessaires à la compréhension des approches modernes de vectorisation d'images. Elle introduit d'abord les principaux modes d'apprentissage, notamment l'apprentissage auto-supervisé, puis décrit les architectures profondes utilisées pour extraire et traiter les caractéristiques visuelles, en particulier les réseaux de neurones, les Transformers et les Vision Transformers.

### 1.3.1 Apprentissage automatique et apprentissage auto-supervisé

L'apprentissage automatique est une discipline du domaine de l'intelligence artificielle dans laquelle, contrairement à l'approche programmatique classique, la machine ne reçoit pas une suite d'instructions détaillées pour résoudre une tâche donnée, car ces instructions seraient souvent trop complexes à définir explicitement [7]. Le principe consiste plutôt à apprendre à partir d'un ensemble de données afin d'améliorer progressivement une performance  $P$  sur une tâche  $T$  à partir d'une expérience  $E$ , selon la définition proposée par Mitchell [8].

Selon la nature des données et du signal de supervision disponibles, plusieurs modes d'apprentissage peuvent être distingués. Dans l'apprentissage supervisé, chaque exemple d'entrée est associé à une cible ou à une étiquette attendue. À l'inverse, dans l'apprentissage non supervisé, le modèle cherche à découvrir des structures présentes dans les données, telles que des groupes d'exemples similaires ou des représentations de plus faible dimension [9].

Cependant, dans le contexte de la vision par ordinateur moderne, ces deux catégories ne suffisent pas toujours à expliquer l'intérêt des modèles pré-entraînés utilisés pour l'extraction de caractéristiques visuelles. En effet, l'annotation manuelle de grands ensembles d'images est une tâche coûteuse et parfois difficile à généraliser à tous les domaines. C'est dans ce cadre que l'apprentissage auto-supervisé prend une place importante. Cette approche exploite des données non étiquetées tout en générant automatiquement un signal de supervision à partir des données elles-mêmes, par exemple en demandant au modèle de prédire une partie manquante, une transformation appliquée ou une relation entre différentes vues d'une même image [10]. Elle permet ainsi d'apprendre des représentations visuelles générales à partir de grandes collections d'images sans annotations humaines explicites, représentations qui peuvent ensuite être transférées vers d'autres tâches de vision par ordinateur [7, 10]. Dans le cadre de la vectorisation d'images, cet aspect est particulièrement pertinent, car l'objectif n'est pas seulement de classer une image, mais d'en extraire une représentation riche de sa structure, de ses régions et de ses détails visuels, afin de faciliter ensuite la prédiction de primitives vectorielles.

### 1.3.2 Apprentissage profond

L'apprentissage profond est une sous-branche de l'apprentissage automatique fondée principalement sur les réseaux de neurones profonds. Ces modèles sont constitués de plusieurs couches de neurones artificiels capables d'apprendre progressivement des représentations de plus en plus abstraites à partir des données. Cette approche s'est fortement développée grâce à l'augmentation de la puissance de calcul, à la disponibilité de grandes quantités de données et aux progrès réalisés dans l'entraînement des

réseaux profonds [7]. Dans le domaine du traitement d'images, les réseaux de neurones convolutionnels (CNN) ont longtemps constitué l'une des architectures dominantes, grâce à leur capacité à exploiter la structure spatiale locale des images et à extraire automatiquement des caractéristiques visuelles telles que les contours, les formes ou les textures [11, 1]. Cependant, les architectures récentes fondées sur l'attention, en particulier les Transformers et les Vision Transformers, ont progressivement élargi cette approche en permettant de modéliser des relations plus globales entre les différentes régions d'une image.

### A. Réseaux de neurones

Les réseaux de neurones artificiels sont des modèles d'apprentissage automatique inspirés, de manière simplifiée, du fonctionnement des neurones biologiques. Ils sont constitués d'unités de calcul appelées neurones artificiels, organisées en couches et reliées entre elles par des connexions pondérées. L'apprentissage consiste à ajuster ces poids à partir des données d'entraînement afin que le réseau puisse produire une sortie correcte pour une tâche donnée [7]. Dans le domaine du traitement d'images, ces modèles sont utilisés pour apprendre automatiquement des fonctions de décision à partir d'exemples, notamment dans les problèmes de classification, de reconnaissance de formes et de reconstruction [1].

### B. Transformers

Les Transformers sont une architecture de réseaux de neurones introduite initialement pour le traitement des séquences, notamment en traduction automatique. Contrairement aux réseaux récurrents, ils ne traitent pas les éléments de la séquence de manière strictement séquentielle, mais s'appuient principalement sur des mécanismes d'attention, en particulier l'auto-attention, afin de modéliser les relations entre les différents éléments d'une entrée [12]. Grâce à leur capacité à apprendre des dépendances complexes et à être entraînés efficacement sur de grandes quantités de données, les Transformers ont ensuite été largement utilisés dans de nombreux domaines de l'apprentissage profond, notamment le traitement du langage naturel, la vision par ordinateur et les modèles multimodaux [7]. Comme l'illustre la figure 1.3, un encodeur Transformer peut être représenté de manière simplifiée par un mécanisme d'auto-attention multi-têtes suivi d'un réseau feed-forward.

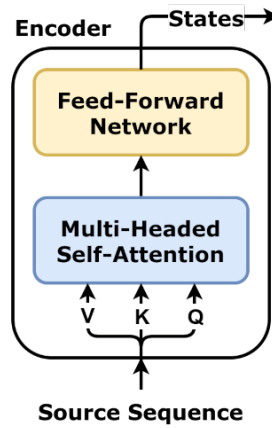


FIGURE 1.3 – Structure simplifiée d'un encodeur Transformer. Figure adaptée de dvgodoy, sous licence CC BY 4.0.

### C. Vision Transformers

Les Vision Transformers, ou ViT, sont une adaptation de l'architecture Transformer au traitement des images. Au lieu de s'appuyer principalement sur des convolutions comme les CNN, une image est divisée en petits blocs appelés patches, par exemple de taille  $16 \times 16$ . Ces patches sont ensuite transformés en vecteurs et traités comme une séquence d'éléments, de manière similaire aux tokens dans le traitement du langage naturel [13]. Grâce au mécanisme d'auto-attention, le modèle peut apprendre des relations globales entre différentes régions de l'image. Cette capacité rend les Vision Transformers particulièrement intéressants pour les tâches où la compréhension de la structure globale de l'image est importante, en complément des informations locales. Comme l'illustre la figure 1.4, les représentations des patches sont combinées à un encodage positionnel, puis transmises à un encodeur Transformer afin de produire la sortie du modèle.

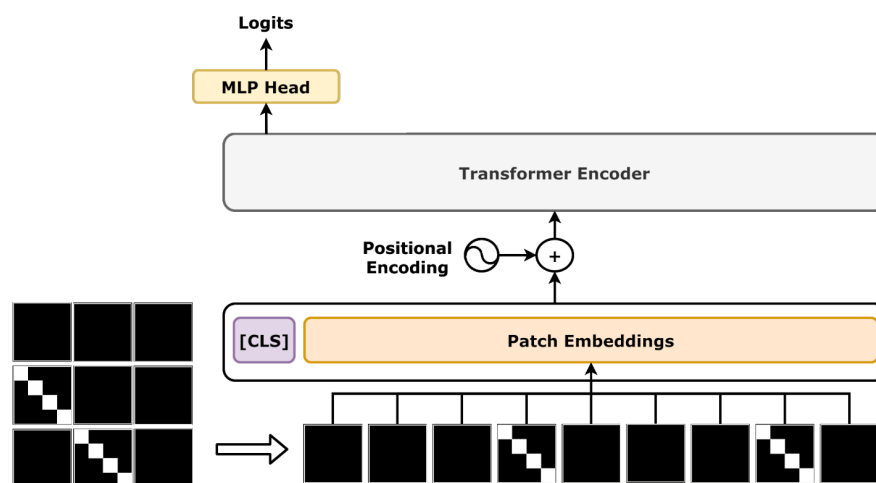


FIGURE 1.4 – Architecture générale d'un Vision Transformer. Figure adaptée de dvgodoy, sous licence CC BY 4.0.

## 1.4 Mécanismes d'attention

Les mécanismes d'attention constituent une composante importante des architectures modernes d'apprentissage profond, en particulier dans les modèles de type Transformer. Leur objectif est de permettre au modèle d'accorder une importance différente aux éléments d'une entrée selon leur pertinence pour la tâche à résoudre, au lieu de traiter toutes les informations de manière uniforme. Ce fonctionnement repose généralement sur trois éléments principaux : une requête, des clés et des valeurs. La requête représente l'information recherchée, les clés servent à mesurer la similarité avec cette requête, et les valeurs contiennent les informations à combiner [12].

Dans le cas de l'attention par produit scalaire mise à l'échelle, les requêtes, les clés et les valeurs sont regroupées respectivement dans les matrices  $Q$ ,  $K$  et  $V$ . Les scores d'attention sont obtenus en calculant le produit entre les requêtes et les clés, puis en le normalisant par  $\sqrt{d_k}$ , où  $d_k$  désigne la dimension des clés. Un masque peut éventuellement être appliqué, notamment lorsque l'on souhaite empêcher le modèle d'accéder à certaines positions. La fonction softmax transforme ensuite, pour chaque requête, les scores obtenus en poids positifs compris entre 0 et 1, dont la somme est égale à 1. Les scores les plus élevés reçoivent ainsi les poids les plus importants, ce qui permet au modèle d'accorder davantage d'attention aux éléments considérés comme les plus pertinents. Ces poids sont finalement utilisés pour effectuer une combinaison pondérée des valeurs [12, 7]. Comme l'illustre la figure 1.5, ce mécanisme enchaîne le produit scalaire entre  $Q$  et  $K$ , la mise à l'échelle, l'application éventuelle d'un masque, la fonction softmax et la combinaison pondérée avec  $V$ .

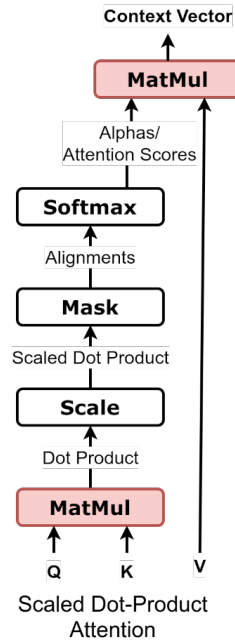


FIGURE 1.5 – Attention par produit scalaire mise à l'échelle. Figure adaptée de dvigodoy, sous licence CC BY 4.0.

Ce mécanisme peut être formalisé par l'équation suivante [12] :

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (1.1)$$

Cette équation montre que les similarités entre les requêtes  $Q$  et les clés  $K$  sont transformées en poids d'attention, puis utilisées pour pondérer les valeurs  $V$ . Le modèle peut ainsi privilégier les informations les plus pertinentes selon le contexte, y compris lorsqu'elles sont éloignées dans la séquence ou dans l'image [12, 7].

### 1.4.1 Self-attention

La self-attention, ou auto-attention, est un cas particulier du mécanisme d'attention dans lequel les requêtes, les clés et les valeurs proviennent de la même séquence. Elle permet donc à chaque élément d'une entrée de prendre en compte les autres éléments de cette même entrée afin de construire une représentation plus riche et contextualisée. Dans une phrase, par exemple, chaque mot peut être relié aux autres mots pour mieux comprendre son sens selon le contexte. Dans une image représentée sous forme de patches, chaque patch peut de la même manière interagir avec les autres régions de l'image [12]. Cette capacité à modéliser les relations internes d'une séquence explique l'importance de la self-attention dans les architectures Transformer. Dans le traitement d'images, cette propriété permet notamment de mettre en relation des régions éloignées d'une même image, contrairement aux opérations strictement locales qui se limitent à un voisinage restreint.

### 1.4.2 Cross-attention

La cross-attention, ou attention croisée, est une variante du mécanisme d'attention dans laquelle les requêtes ne proviennent pas de la même source que les clés et les valeurs. Contrairement à la self-attention, où un élément interagit avec les autres éléments d'une même séquence, la cross-attention permet à une représentation de s'appuyer sur une autre représentation externe.

Dans une architecture Transformer de type encodeur-décodeur, l'encodeur reçoit les données d'entrée et les transforme en représentations internes contenant les informations importantes de cette entrée. Le décodeur utilise ensuite ces représentations afin de générer progressivement la sortie attendue. Lors de la cross-attention, le décodeur produit les requêtes, tandis que les clés et les valeurs proviennent des représentations calculées par l'encodeur. Ce mécanisme permet ainsi au décodeur de sélectionner, à chaque étape de la génération, les informations de l'entrée les plus pertinentes [12, 7].

Dans le contexte de la vectorisation d'images, l'encodeur peut extraire les caractéristiques visuelles de l'image matricielle, tandis que le décodeur génère les paramètres des primitives vectorielles. La cross-attention permet alors aux requêtes associées aux primitives à produire d'interagir avec les clés et les valeurs issues de l'image encodée. Les paramètres vectoriels prédits sont ainsi guidés par les informations visuelles les plus pertinentes, ce qui facilite la génération de primitives adaptées au contenu de l'entrée.

## 1.5 Concepts utilisés dans la vectorisation par apprentissage profond

La vectorisation d'images par apprentissage profond repose sur plusieurs concepts techniques permettant de passer d'une image matricielle, composée de pixels, à une représentation vectorielle décrite par des formes paramétriques. Contrairement aux méthodes classiques de traçage, les approches récentes cherchent à apprendre ou à optimiser automatiquement les paramètres des primitives vectorielles afin de reconstruire au mieux l'image d'entrée. Pour cela, elles utilisent notamment la décomposition de l'image en régions plus simples, le rendu différentiable pour relier les paramètres vectoriels à leur rendu matriciel, ainsi que des fonctions de perte permettant de mesurer l'écart entre l'image originale et l'image reconstruite. Certaines méthodes adoptent également une stratégie progressive de type *coarse-to-fine*, dans laquelle la structure globale de l'image est d'abord reconstruite avant d'ajouter les détails fins [4, 14].

### 1.5.1 Superpixels

Les superpixels sont des régions obtenues en regroupant des pixels voisins présentant des caractéristiques similaires, notamment en termes de couleur et de position spatiale. Ils permettent de remplacer une représentation pixel par pixel par une représentation plus compacte, composée de régions visuellement cohérentes. Parmi les méthodes les plus connues, l'algorithme SLIC (*Simple Linear Iterative Clustering*) adapte le principe du k-means afin de générer efficacement des superpixels. Cette méthode est appréciée pour sa simplicité, sa rapidité, sa faible consommation mémoire et sa capacité à produire des régions qui respectent bien les contours de l'image [3]. Dans le cadre de la vectorisation d'images, les superpixels sont utiles car ils divisent une image complexe en sous-régions plus simples à traiter. SuperSVG exploite précisément cette idée en décomposant l'image d'entrée en superpixels contenant des pixels de couleurs et de textures similaires, puis en vectorisant ces régions de manière progressive [14].

### 1.5.2 Rendu différentiable

Le rendu différentiable est une technique qui permet de relier une représentation vectorielle à son rendu matriciel tout en conservant la possibilité de calculer des gradients. Dans le contexte de la vectorisation d'images, les paramètres des primitives vectorielles, comme les positions des points de contrôle, les couleurs ou l'épaisseur des traits, sont d'abord rendus sous forme d'image raster. L'image obtenue est ensuite comparée à l'image cible à l'aide d'une fonction de perte, puis les gradients de cette perte sont rétropropagés vers les paramètres vectoriels afin de les optimiser [4]. Cette approche permet donc d'utiliser des méthodes d'optimisation ou d'apprentissage profond pour ajuster automatiquement les formes vectorielles à partir d'une supervision dans le domaine des pixels. Elle est notamment utilisée dans des méthodes comme LIVE, qui optimise progressivement des chemins de Bézier dans un cadre de rendu entièrement différentiable afin de produire des SVG organisés en couches [15].

### 1.5.3 Fonctions de perte

Les fonctions de perte jouent un rôle central dans la vectorisation d'images par apprentissage profond, car elles permettent de mesurer l'écart entre l'image matricielle d'origine et l'image reconstruite à partir des primitives vectorielles. Dans les approches utilisant un rendu différentiable, les chemins SVG prédits ou optimisés sont d'abord rasterisés, puis comparés à l'image cible à l'aide d'une fonction de perte calculée dans l'espace des pixels. Une perte simple, comme la MSE, peut être utilisée pour minimiser la différence entre les deux images, mais elle ne suffit pas toujours à préserver correctement la structure, les contours ou la couleur des objets. Par exemple, LIVE montre qu'une

optimisation fondée uniquement sur la MSE peut conduire à un effet de couleur moyenne, puis propose une perte UDF afin de donner plus d'importance aux pixels proches des contours, ainsi qu'une perte Xing pour limiter les auto-intersections des courbes [15]. Dans SuperSVG, les auteurs combinent plusieurs pertes adaptées à la vectorisation par superpixels, notamment une perte de reconstruction normalisée, une perte de frontière pour empêcher les chemins de dépasser les limites du superpixel, et une perte Dynamic Path Warping permettant au modèle de raffinement d'hériter des connaissances du modèle grossier [14].

### 1.5.4 Apprentissage coarse-to-fine

L'apprentissage *coarse-to-fine* désigne une stratégie progressive dans laquelle le modèle commence par reconstruire les éléments globaux ou principaux d'une image, avant d'ajouter progressivement des détails plus fins. Dans le contexte de la vectorisation d'images, cette approche est particulièrement utile car une image complexe est difficile à convertir directement en une représentation SVG précise. En procédant par étapes, le modèle peut d'abord approximer la structure générale, puis améliorer localement la reconstruction. SuperSVG adopte cette logique à travers un cadre d'apprentissage en deux étapes : un modèle grossier prédit d'abord des chemins SVG pour reconstruire la structure principale du superpixel, puis un modèle de raffinement ajoute d'autres chemins afin d'enrichir les détails, en étant guidé par les chemins prédits à l'étape précédente [14]. Une idée similaire apparaît dans LIVE, où l'image est reconstruite progressivement en ajoutant et en optimisant de nouveaux chemins de Bézier dans une représentation en couches, ce qui permet d'obtenir un SVG plus compact et plus cohérent avec la structure visuelle de l'image [15].

## 1.6 Évaluation des résultats

L'évaluation d'une méthode de vectorisation consiste généralement à rendre le SVG généré sous forme matricielle, puis à comparer ce rendu à l'image d'entrée. Cette section présente les principales métriques utilisées dans nos expérimentations : la MSE, le PSNR, la SSIM, la LPIPS et le temps d'inférence.

### 1.6.1 MSE

La MSE (*Mean Squared Error*) mesure l'erreur moyenne entre les pixels de l'image originale  $I$  et ceux de l'image reconstruite  $\hat{I}$  :

$$\text{MSE}(I, \hat{I}) = \frac{1}{HWC} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^C (I_{i,j,c} - \hat{I}_{i,j,c})^2 \quad (1.2)$$

Dans l'équation (1.2),  $H$  et  $W$  représentent les dimensions de l'image et  $C$  son nombre de canaux. Une valeur faible indique une reconstruction plus fidèle au niveau des pixels. Cette métrique reste toutefois limitée, car elle ne correspond pas toujours à la qualité visuelle perçue [15, 14].

### 1.6.2 PSNR

Le PSNR (*Peak Signal-to-Noise Ratio*) est une métrique dérivée de la MSE. Pour des images dont les valeurs sont normalisées entre 0 et 1, il est défini par :

$$\text{PSNR}(I, \hat{I}) = 10 \log_{10} \left( \frac{1}{\text{MSE}(I, \hat{I})} \right) \quad (1.3)$$

Contrairement à la MSE, une valeur élevée du PSNR indique une meilleure fidélité de reconstruction. Cette métrique est principalement utilisée pour évaluer les différences pixel par pixel entre l'image cible et le rendu du SVG [4, 14].

### 1.6.3 SSIM

La SSIM (*Structural Similarity Index Measure*) mesure la similarité structurelle entre deux images en prenant en compte leur luminance, leur contraste et leur structure locale [16]. Elle est définie par :

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (1.4)$$

Dans l'équation (1.4),  $\mu_x$  et  $\mu_y$  désignent les moyennes des deux fenêtres comparées,  $\sigma_x^2$  et  $\sigma_y^2$  leurs variances, et  $\sigma_{xy}$  leur covariance. Les constantes  $C_1$  et  $C_2$  assurent la stabilité du calcul. Une valeur élevée indique une meilleure préservation de la structure visuelle [14].

### 1.6.4 LPIPS

La LPIPS (*Learned Perceptual Image Patch Similarity*) compare deux images à partir de caractéristiques extraites par un réseau de neurones pré-entraîné, plutôt qu'à partir de leurs pixels uniquement [17]. Elle peut être exprimée sous la forme :

$$\text{LPIPS}(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\phi_l(x)_{h,w} - \phi_l(\hat{x})_{h,w})\|_2^2 \quad (1.5)$$

Dans l'équation (1.5),  $\phi_l$  représente les caractéristiques extraites à la couche  $l$  du réseau, tandis que  $w_l$  désigne les poids associés à ces caractéristiques. Une faible valeur correspond à une meilleure similarité perceptuelle entre l'image d'entrée et le rendu SVG [14].

### 1.6.5 Temps d'inférence

Le temps d'inférence correspond à la durée nécessaire pour produire un SVG à partir d'une image d'entrée. Il permet d'évaluer l'efficacité pratique d'une méthode, indépendamment de sa qualité de reconstruction. Les méthodes reposant sur l'optimisation directe des paramètres vectoriels nécessitent généralement plusieurs itérations, tandis que les modèles entraînés peuvent prédire ces paramètres plus rapidement. Cette métrique est donc utilisée conjointement avec les mesures de qualité afin de comparer la précision et la rapidité des différentes approches [14].

## 1.7 Travaux connexes

La vectorisation automatique d'images a connu une évolution progressive, allant des approches algorithmiques classiques jusqu'aux travaux récents combinant apprentissage profond et rendu différentiable. Pour comparer ces travaux, il ne suffit pas de considérer uniquement la similarité visuelle entre l'image d'entrée et le SVG obtenu. Une approche de vectorisation doit également être évaluée selon plusieurs critères, notamment la fidélité visuelle, le temps de calcul, le nombre de chemins générés, la simplicité du SVG, son éditabilité, sa capacité de généralisation et le niveau de contrôle laissé à l'utilisateur [18]. Dans cette section, nous présentons donc les approches les plus pertinentes pour notre sujet, en montrant progressivement comment les limites des approches classiques et fondées sur l'optimisation ont conduit à des travaux plus récents comme SuperSVG.

### 1.7.1 Approches algorithmiques

Les approches algorithmiques reposent généralement sur l'analyse géométrique de l'image, comme l'extraction de contours, la détection de régions ou l'ajustement de courbes. Elles sont souvent rapides et ne nécessitent pas de données d'entraînement, mais restent limitées lorsque l'image contient des couleurs complexes, des textures ou un grand nombre de détails.

#### A. Potrace

Potrace est une méthode classique de vectorisation principalement conçue pour convertir des images binaires en contours vectoriels [19]. Son principe consiste à extraire les frontières entre les zones noires et blanches, à les approximer par des polygones, puis à transformer ces polygones en courbes de Bézier lisses. Cette méthode est simple, rapide et efficace pour des images comme les logos, les textes ou les dessins en noir et blanc. Cependant, elle dépend fortement de la qualité de la binarisation et reste peu adaptée aux images complexes, colorées ou texturées. Potrace constitue donc une

référence importante pour les approches classiques, mais montre aussi les limites des méthodes purement algorithmiques face à une vectorisation plus générale.

### 1.7.2 Approches basées sur l'optimisation

Les approches basées sur l'optimisation cherchent à ajuster directement les paramètres des primitives vectorielles afin que le rendu obtenu soit le plus proche possible de l'image d'entrée. Avec le rendu différentiable, il devient possible de calculer des gradients entre l'image rendue et l'image cible, puis de les utiliser pour modifier les courbes, les couleurs ou l'opacité des chemins SVG.

#### A. DiffVG

DiffVG propose un rasteriseur différentiable permettant de relier le domaine vectoriel au domaine matriciel [4]. Grâce à ce mécanisme, un ensemble de courbes ou de formes vectorielles peut être rendu sous forme d'image, comparé à une image cible, puis optimisé par rétropropagation. Cette approche est importante car elle permet d'utiliser une supervision uniquement matricielle, sans avoir besoin d'un SVG de référence. Elle est donc devenue une base technique pour plusieurs méthodes récentes. Cependant, DiffVG reste coûteux lorsqu'il faut optimiser beaucoup de chemins ou effectuer un grand nombre d'itérations. De plus, le nombre de primitives doit souvent être fixé à l'avance, ce qui limite le contrôle automatique de la complexité du SVG.

#### B. LIVE

LIVE, pour *Layer-wise Image Vectorization*, améliore les approches d'optimisation en introduisant une reconstruction progressive par couches [15]. La méthode ajoute progressivement des chemins de Bézier, puis les optimise afin de reconstruire l'image de manière *coarse-to-fine*. Cette stratégie permet d'obtenir des SVG plus structurés, plus compacts et plus éditables que ceux produits par une optimisation naïve. LIVE introduit également des pertes spécifiques pour mieux préserver les contours et limiter les auto-intersections des chemins. Malgré ces avantages, la méthode reste lente, car chaque image nécessite une optimisation itérative. Le résultat dépend aussi du nombre de chemins choisi et de plusieurs hyperparamètres. LIVE montre donc l'intérêt d'une représentation vectorielle progressive et éditable, mais confirme aussi le besoin de méthodes plus rapides.

### 1.7.3 Approches basées sur l'apprentissage profond

Les approches basées sur l'apprentissage profond cherchent à apprendre directement la relation entre une image matricielle et une représentation vectorielle. Contrairement

aux méthodes d'optimisation qui ajustent les chemins pour chaque image, ces méthodes entraînent un modèle capable de prédire les paramètres vectoriels à partir de l'image d'entrée, ce qui permet généralement une inférence plus rapide.

### A. Im2Vec

Im2Vec propose une méthode de génération de graphiques vectoriels sans supervision vectorielle directe [20]. Contrairement aux approches qui nécessitent des fichiers SVG annotés, le modèle apprend uniquement à partir d'images matricielles. Son architecture suit un schéma encodeur-décodeur : l'image d'entrée est d'abord encodée dans un espace latent, puis ce code latent est utilisé pour générer une représentation vectorielle composée de plusieurs chemins fermés. Pour gérer les formes constituées de plusieurs composants, Im2Vec utilise un module récurrent de type RNN/LSTM qui produit un code latent propre à chaque chemin. Chaque chemin est ensuite décodé sous forme de courbes de Bézier à l'aide d'un décodeur fondé sur des convolutions 1D circulaires, ce qui permet de modéliser des formes fermées comme des déformations d'un cercle.

Le résultat vectoriel généré est ensuite rendu sous forme matricielle à l'aide d'un rendu différentiable, puis comparé à l'image cible afin d'entraîner le modèle sans supervision SVG explicite. Cette approche est intéressante car elle évite le besoin de jeux de données vectoriels annotés, qui sont difficiles à obtenir et souvent ambigus, puisqu'une même image peut être représentée par plusieurs structures vectorielles différentes. Cependant, Im2Vec reste surtout adapté à des images relativement simples ou appartenant à des domaines limités, comme les icônes, les caractères ou les emojis. Il montre donc l'intérêt de l'apprentissage profond pour la génération et la vectorisation de formes paramétriques, mais reste limité pour traiter des images complexes et variées.

### B. SuperSVG

SuperSVG est la méthode la plus proche de notre thématique et constitue une référence importante dans les travaux récents de vectorisation image-to-SVG [14]. Son principe consiste d'abord à décomposer l'image d'entrée en superpixels, afin de réduire la difficulté du problème en travaillant sur des régions plus homogènes en couleur et en texture. Chaque superpixel est ensuite vectorisé séparément par un modèle d'apprentissage profond chargé de prédire les paramètres des chemins SVG correspondants.

L'architecture de SuperSVG repose sur un modèle en deux étapes. Dans la première étape, dite coarse-stage, un encodeur Vision Transformer extrait les caractéristiques visuelles du superpixel. Ces caractéristiques sont ensuite combinées avec des requêtes apprenables associées aux chemins SVG à l'aide d'un module de cross-attention. Un module de self-attention traite ensuite les représentations intermédiaires afin de les projeter

vers l'espace des paramètres vectoriels. Cette première étape permet de reconstruire la structure principale du superpixel. Dans la seconde étape, dite refinement-stage, un second modèle prend en entrée le rendu obtenu à partir des chemins grossiers ainsi que le superpixel cible, puis prédit des chemins supplémentaires destinés à enrichir les détails. Les sorties des deux étapes sont finalement combinées pour produire le SVG final.

SuperSVG combine ainsi plusieurs idées importantes des travaux précédents : le rendu différentiable hérité de DiffVG, la reconstruction progressive proche de LIVE, et la prédiction directe de chemins SVG par apprentissage profond. Cette architecture lui permet d'obtenir un meilleur compromis entre qualité de reconstruction et temps de calcul, notamment par rapport aux méthodes purement basées sur l'optimisation. Néanmoins, ses résultats dépendent encore de la qualité de la décomposition en superpixels, du nombre de chemins utilisés et de la capacité du modèle à représenter correctement les détails fins de chaque région. Malgré ces limites, SuperSVG représente l'approche la plus pertinente pour notre état de l'art, car elle illustre l'évolution récente des méthodes de vectorisation vers des solutions plus rapides, plus précises et mieux adaptées aux images complexes.

## 1.8 Synthèse comparative des approches

Les travaux étudiés montrent que la vectorisation automatique d'images repose sur un compromis entre plusieurs critères : la fidélité visuelle, le temps de calcul, la simplicité du SVG généré, le nombre de chemins, l'éditabilité et la capacité de généralisation. Les méthodes algorithmiques privilégient généralement la rapidité et la simplicité, mais restent limitées aux images peu complexes. Les méthodes basées sur l'optimisation offrent une meilleure flexibilité grâce au rendu différentiable, mais leur coût augmente avec le nombre de chemins et d'itérations. Les approches basées sur l'apprentissage profond cherchent à dépasser cette limite en apprenant à prédire directement les paramètres vectoriels.

TABLE 1.1 – Synthèse comparative des principales approches de vectorisation étudiées.

| Approche | Principe                                                           | Points forts                          | Limites                                                                        | Apport                                        |
|----------|--------------------------------------------------------------------|---------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------|
| Potrace  | Tracé de contours binaires et approximation par courbes de Bézier. | Rapide, simple et sans apprentissage. | Limité aux images complexes, colorées ou texturées; dépend de la binarisation. | Base des approches algorithmiques classiques. |

| Approche | Principe                                                                   | Points forts                                                                       | Limites                                                                   | Apport                                                                      |
|----------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| DiffVG   | Optimisation de primitives SVG via un rendu différentiable.                | Permet une supervision matricielle et la rétropropagation vers les paramètres SVG. | Coûteux, sensible à l'initialisation et au nombre de chemins fixé.        | Introduit le rendu différentiable comme outil central.                      |
| LIVE     | Ajout progressif de chemins de Bézier selon une logique couche par couche. | Produit des SVG plus structurés, compacts et éditables.                            | Temps de calcul élevé ; dépend des chemins et des hyperparamètres.        | Montre l'intérêt d'une reconstruction progressive.                          |
| Im2Vec   | Prédiction de formes vectorielles à partir d'images matricielles.          | Évite la supervision SVG directe et apprend un espace latent.                      | Surtout adapté aux images simples comme les icônes, caractères ou emojis. | Introduit l'apprentissage profond pour prédire des primitives vectorielles. |
| SuperSVG | Vectorisation par superpixels avec reconstruction <i>coarse-to-fine</i> .  | Bon compromis entre qualité, vitesse et traitement d'images complexes.             | Dépend de la décomposition en superpixels et du nombre de chemins.        | Approche récente la plus proche de notre problématique.                     |

Cette comparaison montre que SuperSVG constitue l'approche la plus complète parmi les travaux étudiés. Elle reprend le rendu différentiable introduit par DiffVG, la logique progressive mise en avant par LIVE, et l'efficacité des modèles d'apprentissage profond. Cependant, la vectorisation automatique reste confrontée à plusieurs difficultés, notamment le contrôle de la complexité du SVG, la fidélité visuelle, le temps de calcul et la généralisation à des images variées.

## 1.9 Conclusion

Dans ce chapitre, nous avons présenté les notions essentielles liées à la vectorisation d'images, puis étudié les principales familles de méthodes existantes. L'analyse des travaux connexes montre une évolution progressive des approches algorithmiques vers les méthodes d'optimisation différentiable, puis vers les modèles basés sur l'apprentissage profond.

La suite du mémoire présentera l'approche proposée pour répondre à cette problématique, ainsi que les choix de conception retenus pour la mise en place du système de vectorisation.

## Chapitre 2

# Approche proposée

### 2.1 Introduction

Le chapitre précédent a présenté les notions nécessaires à la compréhension de la vectorisation d’images ainsi que les principales familles de méthodes existantes. Nous décrivons à présent la solution mise en œuvre dans ce projet. Son objectif est de convertir une image matricielle couleur en un ensemble de chemins vectoriels fermés, représentés par des courbes de Bézier cubiques et enregistrés dans un fichier SVG. La méthode ne nécessite pas de fichiers SVG de référence pendant l’entraînement : la supervision est obtenue en rendant les chemins prédits avec DiffVG, puis en comparant le rendu à la région matricielle d’origine.

Dans ce chapitre, nous présentons d’abord la vue d’ensemble de la méthode et la préparation des données. Nous détaillons ensuite l’architecture du modèle, la représentation des primitives vectorielles, le rendu différentiable et les fonctions de perte. Enfin, nous décrivons la stratégie d’entraînement ainsi que le passage des prédictions locales au SVG de l’image complète.

### 2.2 Vue d’ensemble de la méthode

Cette section présente l’organisation générale de l’approche proposée et ses principaux principes de fonctionnement. Nous décrivons d’abord le cheminement suivi depuis la décomposition de l’image en superpixels jusqu’à la prédiction et à l’assemblage des primitives vectorielles. Nous précisons ensuite le mode de supervision, la prédiction des chemins en une seule passe et les principaux modules constituant la pipeline.

### 2.2.1 Principe général

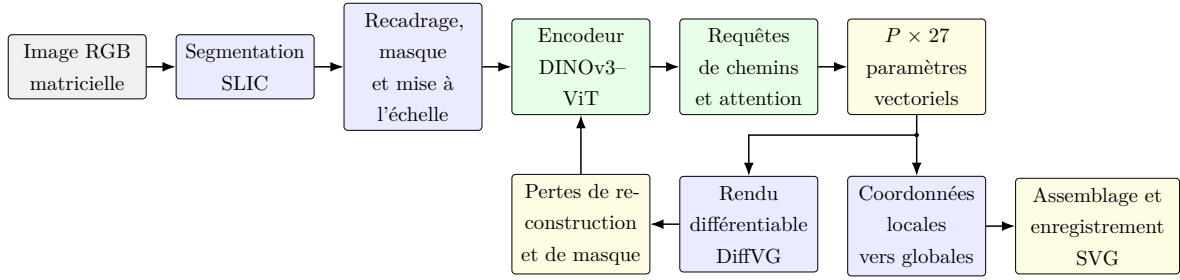


FIGURE 2.1 – Pipeline générale de l'approche proposée, de l'image matricielle aux chemins SVG.

La figure 2.1 donne une vue d'ensemble du flux avant d'en détailler les différentes étapes. Le trajet supérieur est commun à l'entraînement et à l'inférence. Les deux branches inférieures montrent, d'une part, l'utilisation du rendu différentiable pour apprendre les paramètres du réseau et, d'autre part, la transformation géométrique utilisée pour construire le document vectoriel final.

La conversion directe d'une image naturelle complète vers un nombre limité de primitives vectorielles constitue un problème difficile. Une même image peut contenir de nombreux objets, couleurs, contours et détails, tandis qu'il existe plusieurs descriptions SVG capables de produire un rendu visuellement proche. Nous réduisons cette difficulté en travaillant sur des régions dont les pixels sont plus homogènes. Comme les superpixels tendent à suivre les changements de couleur et les frontières locales, chaque prédiction porte sur une forme plus simple que l'image entière. Cette décomposition a été introduite dans la section 1.5.1 ; elle est appliquée ici avec l'algorithme SLIC [3].

Pour chaque région, le réseau prédit un nombre fixé  $P$  de chemins. Chaque chemin est décrit par douze points bidimensionnels et une couleur RGB, soit un vecteur de 27 paramètres. Les chemins sont ensuite rasterisés par DiffVG. Pendant l'entraînement, cette image rendue permet de calculer une erreur de reconstruction et une pénalité lorsque les formes débordent du superpixel. Pendant l'inférence, les coordonnées prédites dans le repère local du recadrage sont replacées dans le repère de l'image originale, puis l'ensemble des chemins est enregistré au format SVG.

### 2.2.2 Caractéristiques de l'approche proposée

L'apprentissage repose uniquement sur des images matricielles. La correspondance entre une image et un SVG n'est pas unique : deux ensembles de courbes différents peuvent produire des rendus proches. Exiger une vérité terrain vectorielle imposerait donc une structure particulière sans garantir qu'elle soit la seule représentation correcte. Le rendu différentiable permet d'éviter cette contrainte en ramenant la supervision dans

l'espace des pixels, comme dans DiffVG et plusieurs méthodes étudiées au chapitre 1 [4, 20].

Les primitives sont prédites en une seule passe du réseau. Contrairement à une optimisation directe qui ajuste les courbes séparément pour chaque nouvelle image, les paramètres sont ici produits par un modèle entraîné. L'inférence ne lance donc pas de descente de gradient sur les chemins. Cette organisation sépare le coût de l'apprentissage, effectué une fois, du traitement des images ultérieures. La version implémentée emploie un seul décodeur, un nombre fixe de chemins par région et deux termes de perte directement liés à la reconstruction et au respect du masque.

Le tableau 2.1 présente les principaux modules et les données qu'ils échangent. Les dimensions indiquées correspondent à une région redimensionnée en  $224 \times 224$  pixels. La dimension interne  $d$  est déterminée automatiquement à partir du modèle DINOv3 chargé.

TABLE 2.1 – Principaux modules de la pipeline proposée.

| Module               | Entrée et sortie                                                 | Rôle dans la pipeline                                                                                   |
|----------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| SLIC                 | Image RGB $\rightarrow$ carte d'étiquettes                       | Décomposer l'image en régions localement homogènes et connexes.                                         |
| Prétraitement        | Région et masque $\rightarrow$ tenseur $3 \times 224 \times 224$ | Recadrer le superpixel, normaliser sa taille et distinguer la région de son arrière-plan.               |
| DINOv3-ViT           | Région $\rightarrow 196 \times d$ tokens                         | Extraire une représentation visuelle dense des $14 \times 14$ patches.                                  |
| Décodeur d'attention | Tokens visuels et $P$ requêtes $\rightarrow P \times d$          | Associer chaque requête de chemin aux zones pertinentes, puis modéliser les interactions entre chemins. |
| Tête linéaire        | $P \times d \rightarrow P \times 27$                             | Prédire les coordonnées normalisées et la couleur de remplissage.                                       |
| DiffVG               | Chemins vectoriels $\rightarrow$ image RGBA                      | Fournir un rendu différentiable pour le calcul des pertes et un aperçu matriciel à l'inférence.         |
| Export SVG           | Chemins globaux $\rightarrow$ fichier SVG                        | Enregistrer les primitives à la résolution de l'image d'origine.                                        |

## 2.3 Préparation des données

Cette section décrit les opérations appliquées aux images avant leur passage dans le modèle. Nous présentons d'abord le jeu de données utilisé, puis la décomposition des

images en superpixels et, enfin, la construction des recadrages locaux fournis au modèle avec leur masque associé.

### 2.3.1 Jeu de données utilisé

L'implémentation utilise le sous-ensemble d'entraînement d'ImageNet comme source d'images naturelles [21]. Les fichiers sont parcourus récursivement et chargés en RGB, puis leurs valeurs entières sont converties en nombres réels dans l'intervalle  $[0, 1]$ . Le chargeur accepte les principaux formats matriciels, notamment JPEG, PNG, BMP, WebP et TIFF. Aucune étiquette de classe ImageNet n'intervient dans l'objectif : seules les images sont exploitées pour apprendre la reconstruction de régions visuelles.

Un exemple du jeu de données ne correspond pas à une image complète. À chaque accès, l'image concernée est segmentée, l'ordre de ses superpixels est mélangé et une région non vide est choisie. Cette stratégie permet de présenter différentes régions d'une même image au cours des époques sans stocker à l'avance tous les recadrages. En cas d'image corrompue ou de région impossible à traiter, le chargeur tente jusqu'à cinq images voisines afin qu'une erreur isolée n'interrompe pas un entraînement long.

### 2.3.2 Décomposition en superpixels

Pour une image  $I \in [0, 1]^{H_0 \times W_0 \times 3}$ , l'algorithme SLIC produit une carte d'étiquettes  $L \in \mathbb{N}^{H_0 \times W_0}$  [3]. Les pixels portant la même étiquette définissent un masque binaire. Le code demande 16 régions pendant l'entraînement, avec une compacité de 10 et un lissage gaussien de paramètre  $\sigma = 1$ . La connectivité est imposée afin d'éviter qu'une même étiquette corresponde à plusieurs composantes éloignées. Le nombre demandé est également borné entre 2 et  $H_0 W_0$  pour conserver une valeur valide sur les petites images.

Ce choix fournit un compromis entre la taille des régions et leur homogénéité. Une valeur faible du nombre de segments laisserait au réseau des régions complexes, alors qu'une valeur très élevée multiplierait le nombre de chemins nécessaires pour reconstruire une image entière. La compacité contrôle quant à elle l'équilibre entre la proximité spatiale et la similarité colorimétrique dans SLIC [3]. Les effets de ces paramètres sur la qualité et le coût de la vectorisation seront évalués séparément dans le chapitre consacré aux expérimentations.

### 2.3.3 Recadrage et construction de l'entrée

Pour un superpixel donné, nous calculons la plus petite boîte englobante  $(x_1, y_1, x_2, y_2)$  contenant tous ses pixels. Dans le repère de l'image, dont l'origine se situe en haut à gauche,  $x$  désigne la coordonnée horizontale et  $y$  la coordonnée verticale. Le couple

$(x_1, y_1)$  correspond au pixel supérieur gauche inclus dans la boîte, tandis que  $x_2$  et  $y_2$  représentent les bornes exclusives à droite et en bas. Le recadrage s'écrit donc  $I[y_1 : y_2, x_1 : x_2]$ , avec une largeur  $x_2 - x_1$  et une hauteur  $y_2 - y_1$ . L'image et le masque sont découpés selon ces mêmes limites. Le recadrage RGB est ensuite redimensionné en  $224 \times 224$  par interpolation bilinéaire, tandis que le masque utilise l'interpolation du plus proche voisin afin de rester binaire. Cette différence est importante : une interpolation continue du masque créerait des valeurs intermédiaires qui rendraient ambiguë l'appartenance à la région.

Soient  $Y \in [0, 1]^{3 \times H \times W}$  le recadrage RGB redimensionné et  $M \in \{0, 1\}^{1 \times H \times W}$  son masque, avec  $H = W = 224$ . Le tenseur fourni au réseau est défini par :

$$X = Y \odot M - (1 - M), \quad (2.1)$$

où le masque est diffusé sur les trois canaux. Les pixels appartenant au superpixel conservent ainsi leur valeur RGB, alors que les pixels extérieurs prennent la valeur  $-1$ . Le réseau reçoit donc à la fois l'apparence de la région et une indication explicite de sa forme. Le tenseur cible reste  $Y$ , mais la perte de reconstruction sera calculée uniquement là où  $M = 1$ . Le masque est conservé séparément pour normaliser cette perte et pénaliser les débordements.

Aucune augmentation par rotation, miroir ou modification colorimétrique n'est appliquée dans la version actuelle du chargeur. La variabilité provient du contenu d'ImageNet et du choix aléatoire du superpixel.

## 2.4 Architecture du modèle

Cette section décrit l'architecture chargée de transformer les caractéristiques visuelles d'un superpixel en paramètres vectoriels. Nous présentons d'abord l'extraction des représentations avec DINOv3, puis le décodage fondé sur des requêtes apprenables, la cross-attention et la self-attention. Enfin, nous détaillons la représentation adoptée pour chaque chemin vectoriel et la manière dont ses paramètres sont interprétés sous forme de courbes de Bézier cubiques.

### 2.4.1 Extraction des caractéristiques avec DINOv3

L'encodeur visuel utilisé est la variante `vit_small_patch16_dinov3`, chargée avec des poids pré-entraînés. DINOv3 est une famille de modèles visuels pré-entraînés par apprentissage auto-supervisé et conçus pour produire des représentations denses transférables [5]. Le recours à un modèle pré-entraîné est adapté à notre problème, car la reconstruction vectorielle nécessite de reconnaître des structures locales, des contours et des relations spatiales sans disposer d'annotations SVG.

Avant l’encodage, chaque canal de  $X$  est standardisé avec les moyennes (0,485, 0,456, 0,406) et les écarts-types (0,229, 0,224, 0,225) employés par l’implémentation. Si la taille de l’entrée diffère de  $224 \times 224$ , une interpolation bilinéaire la ramène à cette résolution. Comme la taille d’un patch est de  $16 \times 16$ , la région est divisée en une grille de  $14 \times 14$ , soit 196 patches. Ce principe de représentation d’une image comme une séquence de patches a été présenté dans la sous-sous-section C. du chapitre 1, consacrée aux Vision Transformers [13].

Le passage dans DINOv3 peut être résumé par :

$$F = E_\theta(X) \in \mathbb{R}^{196 \times d}, \quad (2.2)$$

où  $E_\theta$  désigne l’encodeur visuel et  $d$  sa dimension d’embedding. La sortie de l’encodeur contient également des tokens préfixes, tels que le token de classification et, selon la variante chargée, des tokens registres. Le code retire tous ces préfixes et conserve uniquement les 196 tokens associés aux patches. Il vérifie explicitement leur nombre afin d’éviter de transmettre au décodeur une séquence incompatible avec la résolution prévue.

Les paramètres de DINOv3 ne sont pas gelés pendant l’entraînement. L’optimiseur reçoit l’ensemble des paramètres de l’encodeur de chemins, ce qui inclut le backbone pré-entraîné, les requêtes apprenables, les blocs d’attention et la tête de sortie. Le modèle bénéficie donc de l’initialisation auto-supervisée, puis adapte ses caractéristiques à la prédiction de primitives vectorielles.

## 2.4.2 Requêtes apprenables et cross-attention

Le décodeur maintient  $P$  vecteurs apprenables, appelés requêtes ou tokens de chemins. Ils forment une matrice initiale  $S^{(0)} \in \mathbb{R}^{P \times d}$ , identique pour toutes les images avant son interaction avec les caractéristiques visuelles. Chaque requête est destinée à produire un chemin, mais aucune association fixe n’est imposée entre une requête et une position de l’image. Cette association est apprise par la cross-attention.

Pour chaque tête d’attention, les requêtes proviennent des tokens de chemins, tandis que les clés et les valeurs proviennent des patches DINOv3. En reprenant la notation introduite dans la section 1.4, le calcul s’écrit :

$$Q = \text{LN}(S^{(0)}) W_Q, \quad (2.3)$$

$$K = \text{LN}(F) W_K, \quad V = \text{LN}(F) W_V, \quad (2.4)$$

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right) V, \quad (2.5)$$

où  $d_h$  est la dimension d'une tête. Les sorties des différentes têtes sont concaténées par le module d'attention multi-têtes. Une connexion résiduelle ajoute ensuite l'information obtenue aux requêtes initiales. Le résultat traverse un réseau *feed-forward* composé de deux projections linéaires, d'une activation GELU et d'une dimension cachée quatre fois supérieure à  $d$ . Une seconde connexion résiduelle produit les tokens contextualisés :

$$S^{(1)} = S^{(0)} + \text{MHA}_{\text{cross}}(S^{(0)}, F, F), \quad \tilde{S}^{(1)} = S^{(1)} + \text{FFN}(\text{LN}(S^{(1)})). \quad (2.6)$$

L'implémentation demande huit têtes. Si  $d$  n'est pas divisible par huit, leur nombre est automatiquement diminué jusqu'à obtenir un diviseur valide. Cette précaution permet de conserver le même code avec plusieurs variantes de backbone.

### 2.4.3 Interactions entre les chemins

Après la cross-attention, chaque requête contient principalement l'information visuelle utile à son propre chemin. Une couche de self-attention est ensuite appliquée entre les  $P$  requêtes. Les requêtes, les clés et les valeurs proviennent alors toutes de  $\tilde{S}^{(1)}$ . Cette opération permet à un chemin de tenir compte des autres chemins déjà représentés, ce qui peut limiter la redondance et répartir la reconstruction entre plusieurs primitives.

Le bloc suit la même organisation pré-normalisée que le bloc précédent : LayerNorm, attention multi-têtes, connexion résiduelle, puis réseau *feed-forward* et seconde connexion résiduelle. La version actuelle emploie exactement un bloc de cross-attention et un bloc de self-attention. Elle ne contient ni décodeur récurrent, ni empilement de plusieurs étapes de raffinement.

Une normalisation finale et une couche linéaire transforment chaque token en 27 valeurs. Une fonction sigmoïde borne toutes les sorties entre 0 et 1 :

$$Z = \sigma(W_o \text{LN}(S^{(2)}) + b_o) \in [0, 1]^{P \times 27}. \quad (2.7)$$

Cette contrainte convient simultanément aux coordonnées normalisées et aux trois composantes de couleur. Elle évite qu'un point soit placé hors du canevas local avant l'application de la perte de masque.

### 2.4.4 Représentation d'un chemin vectoriel

Pour le chemin  $p$ , le vecteur prédit est décomposé de la manière suivante :

$$z_p = (x_0, y_0, \dots, x_{11}, y_{11}, r_p, g_p, b_p). \quad (2.8)$$

Les 24 premières valeurs représentent douze points  $P_0, \dots, P_{11}$  dans le repère nor-

malisé du recadrage. Les trois dernières valeurs définissent la couleur de remplissage. DiffVG interprète ces points comme quatre segments de Bézier cubiques reliés bout à bout et ferme le dernier segment sur le point initial. Les points  $P_0$ ,  $P_3$ ,  $P_6$  et  $P_9$  sont les points d’ancrage, tandis que les autres points contrôlent la tangente et la courbure, comme l’illustre la figure 2.2.

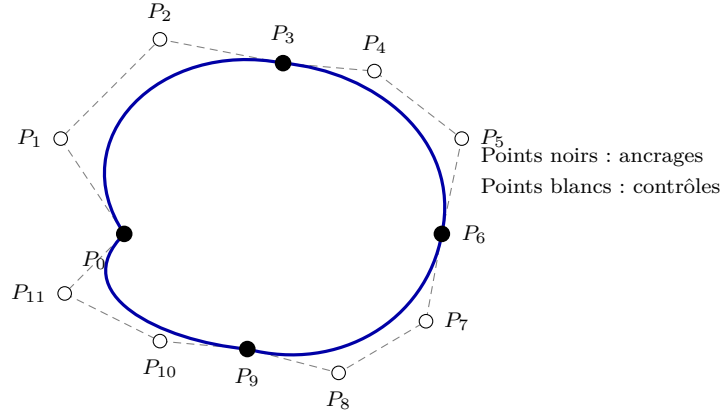


FIGURE 2.2 – Chemin fermé composé de quatre courbes de Bézier cubiques et de douze points prédits.

Pour quatre points consécutifs  $A$ ,  $C_1$ ,  $C_2$  et  $B$ , chaque segment cubique est défini, pour  $t \in [0, 1]$ , par :

$$\mathcal{B}(t) = (1 - t)^3 A + 3(1 - t)^2 t C_1 + 3(1 - t) t^2 C_2 + t^3 B. \quad (2.9)$$

Cette équation complète le rappel sur les courbes de Bézier de la section A. en précisant la structure réellement produite par le modèle. Dans le premier segment, par exemple,  $A = P_0$ ,  $C_1 = P_1$ ,  $C_2 = P_2$  et  $B = P_3$ . Le quatrième segment part de  $P_9$ , utilise  $P_{10}$  et  $P_{11}$  comme points de contrôle, puis revient à  $P_0$  pour fermer la forme.

Les primitives sont remplies avec la couleur  $(r_p, g_p, b_p)$ . Leur opacité est fixée à 1 et leur largeur de contour à 0 ; le modèle produit donc des surfaces colorées sans trait séparé. Le nombre de chemins  $P$  est un hyperparamètre fixé avant la construction du réseau. La valeur par défaut du code est 32, tandis qu’une configuration d’entraînement enregistrée dans le projet utilise 128 chemins par superpixel. Aucune sortie ne prédit actuellement la visibilité, l’opacité ou un symbole d’arrêt permettant de faire varier ce nombre.

## 2.5 Rendu différentiable et fonctions de perte

Cette section présente le mécanisme utilisé pour comparer les primitives vectorielles prédites à la région matricielle cible. Nous décrivons d’abord la construction de la scène et sa rasterisation avec DiffVG, puis la perte de reconstruction calculée à l’intérieur du

superpixel et la pénalité appliquée aux formes qui dépassent son masque. Enfin, nous définissons l’objectif global utilisé pour entraîner l’ensemble du réseau.

### 2.5.1 Construction de la scène DiffVG

Les paramètres  $Z$  sont convertis en objets `Path` et `ShapeGroup` de DiffVG. Pour un canevas local de largeur  $W$  et de hauteur  $H$ , chaque coordonnée normalisée  $(x, y)$  est multipliée par  $(W, H)$ . Les quatre segments reçoivent chacun deux points de contrôle, le chemin est déclaré fermé et sa couleur de remplissage est obtenue à partir des trois dernières valeurs du vecteur. Le rasteriseur différentiable calcule ensuite une image RGBA avec un échantillonnage  $2 \times 2$  par pixel [4].

Si  $\mathcal{R}$  désigne ce rendu, l’image RGB employée pour la reconstruction est composée sur un fond noir :

$$\hat{Y} = \mathcal{R}_{\text{RGB}}(Z) \odot \mathcal{R}_{\alpha}(Z). \quad (2.10)$$

La différentiabilité de  $\mathcal{R}$  permet de transmettre le gradient de l’erreur pixel jusqu’aux coordonnées et aux couleurs, puis à l’ensemble du réseau. DiffVG est exécuté en précision simple, même lorsque la précision mixte est active autour du reste du modèle, afin de conserver un calcul de rendu compatible et stable.

### 2.5.2 Perte de reconstruction normalisée

La première composante de l’objectif mesure l’écart quadratique entre le rendu et la cible à l’intérieur du superpixel. Elle dérive de la MSE définie à l’équation (1.2), mais limite le calcul aux pixels appartenant à la région et normalise leur contribution par la surface du masque, selon le principe de reconstruction par superpixels employé dans SuperSVG [14]. Pour un lot de taille  $B$ , nous définissons :

$$a_i = \frac{1}{HW} \sum_{u=1}^H \sum_{v=1}^W M_{i,u,v}, \quad (2.11)$$

puis :

$$\mathcal{L}_{\text{rec}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{3HW \max(a_i, \frac{1}{HW})} \sum_{c=1}^3 \sum_{u=1}^H \sum_{v=1}^W M_{i,u,v} (\hat{Y}_{i,c,u,v} - Y_{i,c,u,v})^2. \quad (2.12)$$

Cette formulation correspond à une MSE calculée sur les pixels utiles, indépendamment de la taille relative de la région. Sans la division par  $a_i$ , un superpixel occupant une faible partie du recadrage contribuerait moins à l’objectif qu’une région remplissant presque toute sa boîte. La borne inférieure  $1/(HW)$  protège le calcul contre une division

par zéro, même si le prétraitement vérifie déjà que le masque n'est pas vide.

### 2.5.3 Pénalité de débordement

La perte de reconstruction ne contraint pas les pixels extérieurs au masque, puisqu'ils sont multipliés par zéro. Un chemin pourrait donc dépasser largement la région tout en reconstruisant correctement sa partie visible à l'intérieur. Pour limiter ce comportement, les mêmes formes sont rendues une seconde fois en blanc et leur canal alpha est comparé à l'extérieur du superpixel. Cette idée est proche de la perte de frontière proposée dans SuperSVG [14].

Avant le calcul, le masque est dilaté par une opération de maximum local de rayon deux pixels. Soit  $\widetilde{M}_i = \mathcal{D}_2(M_i)$  ce masque dilaté et  $O_i = 1 - \widetilde{M}_i$  la zone extérieure. Si  $\widehat{A}_i$  est le canal alpha du rendu blanc, la pénalité non pondérée vaut :

$$\mathcal{L}_{\text{ext}} = \frac{1}{B} \sum_{i=1}^B \frac{\sum_{u,v} \widehat{A}_{i,u,v} O_{i,u,v}}{\max\left(1, \sum_{u,v} O_{i,u,v}\right)}. \quad (2.13)$$

La dilatation crée une tolérance autour de la frontière. Elle évite de pénaliser trop fortement les pixels d'anticrénelage ou un léger décalage entre le masque redimensionné et le bord de la courbe. La normalisation par la surface extérieure rend la valeur comparable entre des masques de formes différentes.

### 2.5.4 Fonction objective

La fonction objectif combine les deux termes précédents :

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_{\text{masque}} \mathcal{L}_{\text{ext}}. \quad (2.14)$$

Le poids de la pénalité décroît linéairement au fil des époques. Pour une époque  $e \in \{0, \dots, E-1\}$  et  $E > 1$ , il est défini par :

$$\lambda_{\text{masque}}(e) = 0,05 + (0,01 - 0,05) \frac{e}{E-1}. \quad (2.15)$$

La contrainte spatiale est ainsi plus forte au début, lorsque les formes prédites sont encore peu structurées, puis son poids diminue pour laisser davantage de place à l'ajustement des couleurs et des contours. L'objectif ne contient actuellement ni perte perceptuelle LPIPS, ni SSIM, ni terme de contrôle du nombre de chemins. Ces mesures sont utiles pour l'évaluation décrite au chapitre précédent, mais elles ne participent pas à la rétropropagation de la version entraînée.

## 2.6 Stratégie d'entraînement

Cette section décrit le protocole utilisé pour entraîner le modèle et assurer la continuité des expériences. Nous présentons d'abord les choix d'optimisation, l'ordonnancement du taux d'apprentissage et les paramètres techniques retenus, puis le mécanisme de sauvegarde permettant de reprendre un entraînement interrompu et de restaurer automatiquement sa configuration.

### 2.6.1 Optimisation des paramètres

L'entraînement utilise AdamW avec un taux d'apprentissage initial de  $10^{-4}$  et une décroissance de poids de 0,05. Le taux d'apprentissage augmente d'abord linéairement pendant une phase de *warm-up*, puis suit une décroissance cosinus jusqu'à la fin de l'entraînement. Tous les paramètres de l'encodeur de chemins sont optimisés conjointement. La graine aléatoire est appliquée à Python, NumPy, PyTorch et CUDA afin de rendre les tirages aussi reproductibles que possible.

La précision mixte automatique est activée lorsque l'entraînement s'exécute sur GPU. Un facteur d'échelle dynamique est appliqué à la perte avant la rétropropagation, puis l'optimiseur et l'ordonnanceur sont mis à jour à chaque lot. Le code permet une limitation optionnelle de la norme du gradient, mais aucune valeur n'est définie dans les configurations enregistrées ; cette limitation est donc inactive pour les entraînements concernés.

Le tableau 2.2 reprend la configuration complète enregistrée pour l'entraînement à 128 chemins. Il décrit le protocole technique, sans anticiper l'analyse des performances qui appartient au chapitre suivant.

TABLE 2.2 – Configuration enregistrée pour l’entraînement du modèle à 128 chemins par superpixel.

| Paramètre                        | Valeur                                                      |
|----------------------------------|-------------------------------------------------------------|
| Source des images                | ImageNet, avec un maximum de 10 000 images                  |
| Taille d’une région              | $224 \times 224$ pixels                                     |
| Paramètres SLIC à l’entraînement | 16 régions, compacité 10, $\sigma = 1$                      |
| Backbone                         | <code>vit_small_patch16_dinov3</code> , poids pré-entraînés |
| Nombre de chemins par région     | 128                                                         |
| Taille des lots                  | 16 régions                                                  |
| Nombre d’époques                 | 20                                                          |
| Optimiseur                       | AdamW, taux $10^{-4}$ , décroissance de poids 0,05          |
| Ordonnancement                   | <i>Warm-up</i> de 4 époques, puis décroissance cosinus      |
| Précision mixte                  | Activée sur CUDA                                            |
| Graine aléatoire                 | 1234                                                        |
| Poids de la perte de masque      | Décroissance linéaire de 0,05 à 0,01                        |

Les valeurs du tableau 2.2 décrivent la configuration effectivement entraînée et ne sont pas présentées comme l’optimum d’une recherche exhaustive. Elles résultent des contraintes de l’architecture, du budget de calcul disponible et de la nécessité de conserver un entraînement reproductible. La limite de 10 000 images maintient une diversité de scènes naturelles tout en bornant la durée de l’expérience. La résolution  $224 \times 224$  correspond à l’entrée utilisée par le backbone DINOv3 et produit exactement 196 patches de taille  $16 \times 16$ . Les paramètres SLIC restent identiques à ceux du prétraitement : 16 régions réduisent la complexité locale sans multiplier excessivement les recadrages, tandis que la compacité 10 et le lissage  $\sigma = 1$  favorisent des régions spatialement régulières sans effacer fortement les frontières.

La variante `vit_small_patch16_dinov3` a été retenue afin d’exploiter des représentations denses pré-entraînées tout en limitant le coût de l’entraînement local. Le budget de 128 chemins par région définit la configuration finale la plus expressive évaluée dans ce travail ; il quadruple le nombre de primitives par rapport à la configuration initiale à 32 chemins, sans modifier la structure du décodeur. Les conséquences de cette augmentation sur la fidélité, le temps de rendu et la taille du SVG sont mesurées au chapitre 3. La taille de lot de 16 régions permet quant à elle de paralléliser les recadrages tout en respectant la mémoire disponible sur le GPU utilisé.

Le taux d'apprentissage de  $10^{-4}$  et la décroissance de poids de 0,05 fournissent des mises à jour modérées pour l'ajustement conjoint du backbone pré-entraîné et du décodeur. Le *warm-up* de quatre époques limite les variations brusques au début de cette adaptation, puis la décroissance cosinus réduit progressivement le taux jusqu'à la vingtième époque, qui correspond au budget d'entraînement retenu. La précision mixte diminue le coût mémoire des opérations compatibles, tandis que la graine 1234 fixe les principales sources d'aléa. Enfin, la décroissance du poids de masque de 0,05 à 0,01 impose d'abord plus fortement le respect des frontières, puis accorde progressivement davantage d'importance à la reconstruction intérieure.

### 2.6.2 Sauvegarde et reprise

À la fin de chaque époque, le programme enregistre l'état du modèle, de l'optimiseur, de l'ordonnanceur et du facteur d'échelle de la précision mixte. Les arguments de l'expérience sont également conservés dans le point de contrôle et dans un fichier JSON séparé. Deux formes de sauvegarde sont produites : un fichier numéroté pour chaque époque et un fichier `latest.pt` représentant le dernier état écrit.

Cette organisation permet de reprendre un entraînement sans réinitialiser le taux d'apprentissage ni les statistiques de l'optimiseur. Lors d'une reprise, l'époque suivante est calculée à partir du numéro sauvegardé et tous les états sont restaurés. Elle permet également au programme d'inférence de reconstruire automatiquement le nombre de chemins, la taille du canevas et le nom du backbone associés au modèle chargé.

## 2.7 Inférence et génération du SVG

Cette section décrit le passage du modèle entraîné à la génération du fichier SVG final. Nous présentons d'abord le traitement de l'image complète et la transformation des coordonnées locales en coordonnées globales, puis l'algorithme d'inférence utilisé pour assembler les chemins prédits. Nous comparons ensuite les flux d'entraînement et d'inférence avant de préciser les limites de la version actuellement implémentée.

### 2.7.1 Traitement de l'image complète

Pendant l'entraînement, une seule région aléatoire est extraite de chaque image chargée. À l'inférence, l'objectif est au contraire de couvrir l'image entière. SLIC est donc appliqué une fois, puis toutes les étiquettes non vides sont parcourues. La configuration par défaut demande 64 régions, avec la même compacité 10 et le même  $\sigma = 1$ . Chaque région est préparée de la même manière qu'à l'entraînement et les recadrages sont regroupés par lots de 16 avant leur passage dans le réseau.

Le modèle retourne des coordonnées normalisées dans la boîte englobante de chaque superpixel. Pour un point prédit  $(u, v) \in [0, 1]^2$  et une boîte  $(x_1, y_1, x_2, y_2)$ , le passage au repère de l'image originale est effectué par :

$$x = x_1 + (x_2 - x_1)u, \quad y = y_1 + (y_2 - y_1)v. \quad (2.16)$$

Cette transformation est appliquée aux douze points de chaque chemin, tandis que la couleur reste inchangée. Les chemins de toutes les régions sont ensuite concaténés. Si SLIC produit effectivement  $R$  régions et que le modèle utilise  $P$  requêtes, le SVG contient donc  $R \times P$  chemins. La version actuelle ne supprime aucun chemin après la prédiction et ne fusionne pas les formes similaires.

## 2.7.2 Algorithme d'inférence

L'algorithme 1 résume les opérations réalisées. Le rendu PNG est produit à la résolution originale pour permettre une inspection rapide. Le fichier SVG est enregistré à partir des mêmes chemins globaux, sans conversion intermédiaire en pixels.

---

**Algorithm 1** Vectorisation d'une image complète et génération du SVG.

---

**Require:** Image RGB  $I$ , modèle entraîné  $f_\theta$ , nombre de régions  $R$

**Ensure:** Fichier SVG et aperçu PNG

```

1:  $L \leftarrow \text{SLIC}(I, R)$ 
2:  $S_{\text{global}} \leftarrow \emptyset$ 
3: for all étiquettes  $k$  présentes dans  $L$  do
4:    $M_k \leftarrow [L = k]$ 
5:    $(X_k, b_k) \leftarrow \text{PRÉPARERRÉGION}(I, M_k)$ 
6:    $S_k \leftarrow f_\theta(X_k)$ 
7:    $S_k^g \leftarrow \text{LOCALVERSGLOBAL}(S_k, b_k)$ 
8:    $S_{\text{global}} \leftarrow S_{\text{global}} \cup S_k^g$ 
9: end for
10:  $\hat{I} \leftarrow \text{DIFFVG}(S_{\text{global}}, \text{taille}(I))$ 
11:  $\text{ENREGISTRERPNG}(\hat{I})$ 
12:  $\text{ENREGISTRERSVG}(S_{\text{global}}, \text{taille}(I))$ 

```

---

En pratique, les régions sont traitées par lots plutôt qu'une par une comme dans ce pseudo-code, mais cette optimisation ne change pas le résultat. Le calcul est exécuté sans gradient et peut utiliser la précision mixte sur GPU. Lorsqu'un point de contrôle est chargé, les paramètres architecturaux qu'il contient sont prioritaires, sauf si l'utilisateur fournit explicitement une valeur différente sur la ligne de commande.

### 2.7.3 Différences entre entraînement et inférence

Le tableau 2.3 met en évidence les différences qui découlent du rôle de chaque phase. Le même prétraitement local et le même réseau sont conservés, mais la sélection des régions et l'utilisation des sorties changent.

TABLE 2.3 – Comparaison du flux d'entraînement et du flux d'inférence.

| Élément                 | Entraînement                                      | Inférence                                      |
|-------------------------|---------------------------------------------------|------------------------------------------------|
| Segmentation par défaut | 16 superpixels demandés par image                 | 64 superpixels demandés par image              |
| Régions traitées        | Une région aléatoire par image et par accès       | Toutes les régions non vides de l'image        |
| Repère des points       | Coordonnées locales normalisées                   | Coordonnées replacées dans l'image originale   |
| DiffVG                  | Produit le rendu utilisé par les pertes           | Produit uniquement l'aperçu PNG                |
| Sortie vectorielle      | Pas de fichier SVG dans la boucle d'apprentissage | Tous les chemins sont assemblés et enregistrés |
| Gradient                | Activé jusqu'au backbone DI-NOv3                  | Désactivé                                      |

### 2.7.4 Périmètre de la version implémentée

La méthode mise en œuvre couvre la chaîne complète, depuis une image RGB jusqu'à un fichier SVG éditable, mais certains choix délimitent son comportement. Le nombre de chemins est fixé pour toutes les régions, indépendamment de leur complexité. Une zone presque uniforme reçoit donc autant de primitives qu'une zone contenant plusieurs variations de couleur. De plus, les chemins sont toujours fermés, opaques et remplis ; les traits ouverts, les contours d'épaisseur variable, les dégradés et la transparence ne sont pas représentés.

La séparation en superpixels rend le problème local plus accessible, mais elle rend également le résultat dépendant de SLIC. Une frontière mal placée peut répartir un même objet entre plusieurs recadrages. La perte de masque encourage les formes à rester dans leur région, sans toutefois appliquer un découpage géométrique strict au moment de l'export. Enfin, la version actuelle ne contient pas l'étape de raffinement, la perte de transfert entre étapes ou l'optimisation finale des courbes présentes dans la méthode SuperSVG complète [14].

Ces éléments ne constituent pas des résultats expérimentaux ; ils décrivent le périmètre exact du système évalué. Leur influence sur la fidélité du rendu, le nombre de

chemins et le temps d'inférence devra être mesurée dans le chapitre suivant à l'aide des métriques présentées au chapitre 1.

## 2.8 Conclusion

Dans ce chapitre, nous avons présenté l'approche proposée pour la vectorisation automatique d'images matricielles. La méthode décompose d'abord l'image avec SLIC, prépare chaque superpixel dans un repère local, puis utilise un encodeur DINOv3 et un décodeur d'attention pour prédire des chemins fermés composés de courbes de Bézier cubiques. Chaque primitive est décrite par douze points de contrôle et une couleur de remplissage.

L'apprentissage ne nécessite pas de SVG cible. DiffVG transforme les prédictions en images et permet d'optimiser une perte de reconstruction normalisée ainsi qu'une pénalité de débordement hors du masque. À l'inférence, les chemins locaux sont replacés dans le repère de l'image originale, assemblés et enregistrés directement au format SVG. Le chapitre suivant sera consacré au protocole expérimental et à l'évaluation quantitative et qualitative de cette architecture.

## Chapitre 3

# Expérimentations et Résultats

### 3.1 Introduction

Dans ce chapitre nous présentons d’abord l’environnement matériel et logiciel, le jeu de test et les conditions du benchmark. Après les tests de validation et l’étude de l’évolution de l’apprentissage, nous comparons quantitativement et qualitativement les quatre approches. Nous revenons ensuite sur les deux configurations de notre architecture, puis discutons les principaux enseignements et les limites de l’évaluation.

### 3.2 Environnement expérimental

Les résultats expérimentaux dépendent à la fois des ressources de calcul et des configurations logicielles employées. Nous précisons donc le matériel utilisé avant de décrire les deux configurations de notre approche qui seront évaluées dans la suite du chapitre.

#### 3.2.1 Configuration matérielle et logicielle

Les entraînements et les évaluations ont été exécutés localement sous Windows 11. Le calcul principal repose sur une carte graphique NVIDIA GeForce RTX 4070 SUPER disposant de 12 282 Mo de mémoire vidéo. Le tableau 3.1 récapitule les composants utilisés. Les temps présentés dans la suite dépendent de cet environnement et ne doivent donc pas être interprétés comme des valeurs indépendantes du matériel.

TABLE 3.1 – Environnement matériel et logiciel utilisé pour les expérimentations.

| Élément                | Configuration                                                    |
|------------------------|------------------------------------------------------------------|
| Système d'exploitation | Windows 11, version 10.0.26200, architecture 64 bits             |
| Processeur             | AMD Ryzen 7 7800X3D, 8 cœurs et 16 processeurs logiques          |
| Mémoire vive           | 31,1 Go                                                          |
| Carte graphique        | NVIDIA GeForce RTX 4070 SUPER, 12 282 Mo, pilote 591.86          |
| Environnement Python   | Python 3.11.15                                                   |
| Calcul profond         | PyTorch 2.5.1, CUDA 12.4, précision mixte automatique            |
| Vision et modèle       | timm 1.0.27, scikit-image 0.26.0, Pillow 11.1.0                  |
| Rendu et métriques     | pydiffvg avec extension native CUDA, LPIPS avec backbone AlexNet |

PyTorch détecte correctement le GPU et DiffVG utilise son extension CUDA. La précision mixte est activée pour le passage dans DINOv3 et le décodeur, tandis que le rendu différentiable reste calculé en précision simple, conformément à l'implémentation décrite au chapitre 2. Les mesures LPIPS sont calculées séparément sur le processeur et ne sont pas incluses dans le temps d'inférence.

Les dépôts de référence ont été exécutés dans des environnements Conda isolés sur la même machine et la même carte graphique. SuperSVG et Im2Vec utilisent PyTorch 2.5.1, tandis que LIVE utilise PyTorch 2.4.1 ; chaque environnement dispose de pydiffvg avec prise en charge de CUDA. Cette séparation conserve les dépendances attendues par les implémentations tout en maintenant un matériel commun pour les mesures temporelles.

### 3.2.2 Configurations de notre approche

Les deux checkpoints évalués utilisent le même backbone `vit_small_patch16_dinov3`, une entrée locale de  $224 \times 224$  pixels, SLIC avec 16 régions demandées, ainsi que l'optimiseur AdamW. Dans les deux cas, les paramètres de DINOv3 sont ajustés pendant l'entraînement : ils ne sont pas gelés. Le backbone contient 21 586 944 paramètres et fait partie de l'ensemble transmis à l'optimiseur.

Le tableau 3.2 montre toutefois que les budgets d'entraînement diffèrent. La configuration à 32 chemins utilise 5 000 images pendant 10 époques, avec une limite de 200 lots par époque. Celle à 128 chemins utilise 10 000 images pendant 20 époques et parcourt 625 lots par époque. Le second modèle dispose donc à la fois de davantage

de primitives et de davantage d'étapes d'optimisation. Les checkpoints des dixième et vingtième époques sont respectivement retenus pour représenter les deux configurations finales. L'étude interne compare ainsi des configurations complètes et ne constitue pas une ablation isolant le seul nombre de chemins.

TABLE 3.2 – Configurations finales comparées pendant l'évaluation.

| Paramètre                 | Configuration 32 | Configuration 128 |
|---------------------------|------------------|-------------------|
| Chemins par superpixel    | 32               | 128               |
| Images d'entraînement     | 5 000            | 10 000            |
| Nombre d'époques          | 10               | 20                |
| Lots par époque           | 200 au maximum   | 625               |
| Taille des lots           | 16               | 16                |
| Époques de <i>warm-up</i> | 6                | 4                 |
| Paramètres du modèle      | 25 160 091       | 25 196 955        |
| Paramètres entraînaibles  | 25 160 091       | 25 196 955        |
| Checkpoint évalué         | Époque 10        | Époque 20         |

L'écart de 36 864 paramètres entre les deux réseaux provient principalement des 96 requêtes de chemins supplémentaires. Le coût du backbone et des blocs d'attention reste presque identique. En revanche, le nombre de primitives transmises à DiffVG est multiplié par quatre, ce qui aura une influence plus importante sur le rendu et l'export du SVG que sur le passage dans le réseau.

### 3.3 Benchmark et conditions d'évaluation

Le benchmark repose sur des images absentes des entraînements de notre approche et sur des conditions de rendu communes. Nous détaillons d'abord la constitution des deux sous-ensembles évalués, puis les conditions d'inférence, les métriques de fidélité et les mesures de complexité vectorielle.

#### 3.3.1 Construction du jeu de test

Les images d'entraînement sont parcourues dans un ordre déterministe avant l'application de leur limite. Le jeu interne réunit donc 12 fichiers situés après les 10 000 premiers éléments de cet ordre, ce qui le rend disjoint des deux ensembles d'entraînement. Issu d'ImageNet sans utiliser les étiquettes de classe, il contient des animaux, des personnes, des objets et des scènes présentant différents niveaux de texture [21].

La comparaison entre approches retient quatre de ces images : une scène complexe, deux objets sur un fond presque uniforme, une salamandre entourée d'une texture

végétale et une scène sombre peu texturée. Ce sous-ensemble rend possible l’exécution complète de LIVE. Les quatre entrées sont redimensionnées en  $256 \times 256$  afin de fournir les mêmes pixels aux implémentations à canevas carré. Pour l’étude interne, les 12 images conservent au contraire leur rapport d’aspect et leur côté le plus long mesure 256 pixels.

L’étude interne utilise les paramètres d’entraînement de SLIC, soit 16 régions demandées, une compacité de 10 et  $\sigma = 1$ . Après le contrôle de connectivité, le nombre effectif varie de 5 à 13, avec une moyenne de  $11,08 \pm 2,39$  régions. Le nombre final de chemins dépend de ce nombre effectif et du nombre de requêtes du modèle. Dans l’exemple de la figure 3.1, les 16 régions demandées deviennent 5 régions connexes ; la dernière vue les représente par leur couleur moyenne.

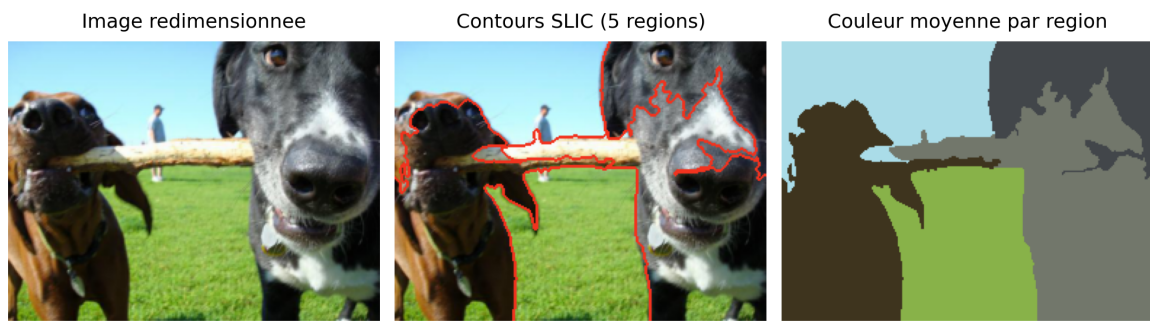


FIGURE 3.1 – Exemple de segmentation SLIC appliquée à une image du jeu de test.

### 3.3.2 Conditions d’inférence

Pour l’étude interne, les deux modèles reçoivent les mêmes images et les mêmes paramètres SLIC. Les régions sont traitées par lots de 16 en précision mixte, puis rendues par DiffVG avec un échantillonnage  $2 \times 2$  par pixel. Une phase de *warm-up* précède la mesure et CUDA est synchronisé autour de chaque étape chronométrée. Le temps total additionne la préparation, la prédiction, le rendu et l’écriture du SVG ; il exclut le chargement du checkpoint et de l’image, l’aperçu PNG, la validation et le calcul des métriques. Les valeurs correspondent à la moyenne des 12 images.

Pour la comparaison externe, chaque approche produit son SVG avec les réglages de la section 3.6. Les documents sont relus par la même version de pydiffvg, rendus avec un échantillonnage  $2 \times 2$ , puis ramenés à  $256 \times 256$  pour les métriques. Les conventions de fond propres aux implémentations sont conservées. Le chargement des poids, la rasterisation commune et le calcul des métriques restent exclus du temps de vectorisation.

### 3.3.3 Métriques et complexité vectorielle

Nous utilisons les quatre métriques introduites au chapitre 1. La MSE et le PSNR mesurent la fidélité au niveau des pixels, la SSIM mesure la conservation de la structure locale [16], et la LPIPS compare des caractéristiques perceptuelles extraites par AlexNet [17]. Une MSE et une LPIPS faibles sont préférables, tandis qu'un PSNR et une SSIM élevés indiquent une meilleure reconstruction. Selon l'étude, les résultats sont donnés sous la forme moyenne  $\pm$  écart-type sur les 12 images internes ou sur les quatre images communes.

La similarité matricielle ne suffit toutefois pas à caractériser un résultat vectoriel. Nous enregistrons également le nombre de chemins, le nombre de segments de Bézier cubiques, la taille du fichier SVG et le temps de vectorisation. Le nombre de segments complète le nombre de balises `path`, puisqu'un chemin peut regrouper plusieurs courbes cubiques. La taille du fichier fournit une indication pratique du coût de stockage et d'édition. Pour l'étude interne, nous conservons en plus le détail des étapes temporelles et le pic de mémoire GPU supplémentaire au-delà du modèle déjà chargé.

## 3.4 Validation technique de l'implémentation

Avant l'analyse des performances, deux vérifications portent sur les mécanismes dont dépend l'évaluation. La première contrôle la transmission des gradients à travers DiffVG ; la seconde vérifie l'intégrité des checkpoints, des rendus et des documents SVG produits.

### 3.4.1 Test du rendu différentiable

Nous avons vérifié la coopération entre PyTorch, CUDA et DiffVG en optimisant pendant 200 étapes le centre, le rayon et la couleur d'un cercle afin de reproduire une cible. La présence et la finitude des gradients sont contrôlées pendant ce test. La MSE passe de 0,061617 à 0,000307 et la géométrie obtenue rejoint étroitement celle de la cible. Ce résultat ne mesure pas la qualité de la vectorisation ; il vérifie la rétropropagation depuis l'image rendue vers les paramètres géométriques et colorimétriques [4].

### 3.4.2 Validation des checkpoints et des SVG

Les deux checkpoints sont chargés avec des paramètres compatibles avec leur architecture. Pour chaque image, nous contrôlons la finitude des prédictions et des pixels rendus, puis le bon format XML du SVG et sa relecture avec `pydiffvg.svg_to_scene`. Cette dernière vérification doit conserver les dimensions du canevas et les chemins attendus.

Le tableau 3.3 résume les contrôles exécutés. Les 24 sorties correspondent aux 12 images traitées par les deux configurations. Toutes les validations passent, ce qui confirme que les métriques sont calculées sur des rendus associés à des documents SVG effectivement rechargeables.

TABLE 3.3 – Résultats des tests techniques de l’implémentation.

| Test                               | Résultat attendu                    | Résultat obtenu                 |
|------------------------------------|-------------------------------------|---------------------------------|
| Rétropropagation DiffVG sur CUDA   | Gradients présents et finis         | Réussi pour tous les paramètres |
| Optimisation du cercle, 200 étapes | Diminution de la perte              | 0,061617 $\rightarrow$ 0,000307 |
| Chargement strict des checkpoints  | Aucun paramètre absent ou inattendu | 2/2 checkpoints valides         |
| Finitude des sorties               | Aucun NaN ou in fini                | 24/24 rendus valides            |
| Analyse XML des SVG                | Document bien formé                 | 24/24 fichiers valides          |
| Relecture avec DiffVG              | Dimensions et chemins conservés     | 24/24 scènes rechargées         |

### 3.5 Évolution de l’apprentissage

Les journaux détaillés de chaque lot n’ayant pas été conservés sous une forme structurée, nous ne reconstruisons pas artificiellement une courbe de perte d’entraînement. Nous évaluons plutôt cinq checkpoints cohérents de la configuration à 32 chemins sur le même jeu de test. Les époques 1, 3, 5, 7 et 10 sont ainsi comparées avec exactement les mêmes conditions.

TABLE 3.4 – Évolution des métriques moyennes du modèle à 32 chemins.

| Époque | MSE ↓   | PSNR ↑ | SSIM ↑ | LPIPS ↓ |
|--------|---------|--------|--------|---------|
| 1      | 0,05351 | 13,49  | 0,311  | 0,663   |
| 3      | 0,03173 | 15,78  | 0,359  | 0,618   |
| 5      | 0,02364 | 17,19  | 0,368  | 0,563   |
| 7      | 0,01961 | 18,14  | 0,408  | 0,530   |
| 10     | 0,01615 | 19,13  | 0,438  | 0,492   |

La figure 3.2 met en évidence une amélioration monotone des quatre métriques mesurées. Entre la première et la dixième époque, la MSE passe de 0,05351 à 0,01615,

soit une diminution absolue de 0,03736, tandis que le PSNR gagne 5,64 dB. La SSIM passe de 0,311 à 0,438, soit une augmentation de 0,127, et la LPIPS diminue de 0,663 à 0,492, soit un écart favorable de 0,171. La baisse la plus forte de la MSE apparaît pendant les premières époques, puis les gains deviennent plus progressifs. Cette évolution est cohérente avec un apprentissage effectif de la reconstruction par les requêtes de chemins.

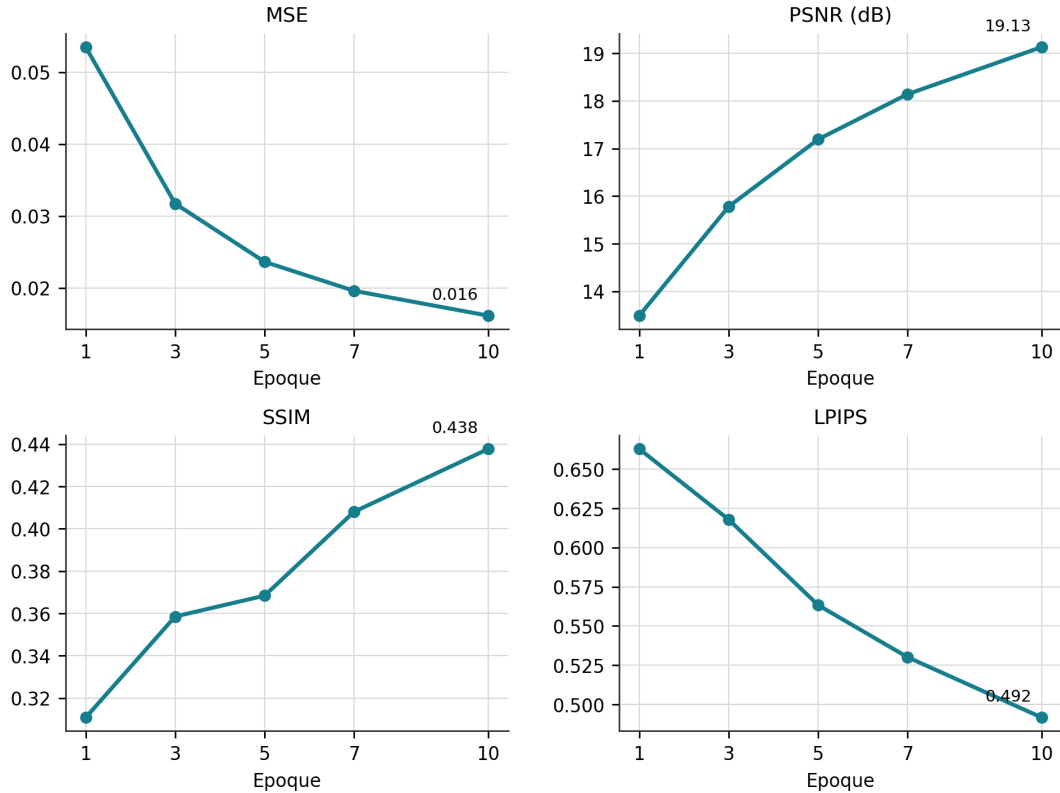


FIGURE 3.2 – Évolution des métriques sur le jeu de test pour la configuration à 32 chemins.

Ces valeurs ne permettent pas d'affirmer que l'entraînement est totalement convergé à l'époque 10. Les métriques continuent à progresser entre les deux derniers points observés. Elles montrent néanmoins une trajectoire régulière sur des images non utilisées pour l'optimisation et fournissent une validation rétrospective de l'apprentissage.

### 3.6 Comparaison avec les approches de référence

La comparaison externe situe notre système par rapport à trois approches représentatives des travaux connexes. Nous précisons leurs configurations avant d'examiner la fidélité, le temps de vectorisation, la complexité des SVG et les différences visibles, puis nous délimitons la portée de ce benchmark.

### 3.6.1 Configurations retenues

Nous comparons notre modèle final à SuperSVG, LIVE et Im2Vec, présentés dans les travaux connexes du chapitre 1. Les trois dépôts de référence ont été exécutés localement et aucun score publié dans les articles n’est recopié dans nos tableaux. Chaque valeur provient ainsi des mêmes quatre images et de la même procédure de rasterisation. Les configurations retenues sont résumées dans le tableau 3.5.

Notre approche utilise le checkpoint de la vingtième époque, 128 requêtes par superpixel et 16 régions SLIC demandées. SuperSVG est chargé avec ses poids pré-entraînés grossiers et de raffinement ; le nombre cible est fixé à 1 000 chemins et l’optimisation finale optionnelle est désactivée. Cette configuration mesure donc la prédiction *coarse-to-fine* du modèle sans descente de gradient supplémentaire sur chaque image [14]. LIVE utilise l’expérience progressive à 32 chemins. Les groupes 1, 2, 4, 8, 16 puis 1 sont ajoutés successivement, avec 500 itérations par étape, soit 3 000 étapes d’optimisation par image [15].

Le checkpoint Im2Vec disponible dans le projet a été entraîné sur des emojis. Il produit quatre chemins superposés, chacun composé de vingt segments cubiques, à partir d’une entrée interne de  $128 \times 128$  pixels [20]. Nous l’incluons afin d’observer le comportement réel de cette représentation compacte hors de son domaine ; ses résultats sur des photographies ImageNet ne représentent donc pas ceux d’un modèle Im2Vec réentraîné sur des images naturelles.

TABLE 3.5 – Configurations utilisées pour la comparaison entre approches.

| Approche       | Mode de vectorisation                   | Configuration évaluée                                               | Budget vectoriel                      |
|----------------|-----------------------------------------|---------------------------------------------------------------------|---------------------------------------|
| Notre approche | Prédiction directe par superpixels      | Modèle final, 16 régions SLIC demandées, 128 requêtes par région    | Variable selon les régions effectives |
| SuperSVG       | Prédiction <i>coarse-to-fine</i>        | Poids officiels, cible de 1 000 chemins, aucune optimisation finale | 1 000 chemins                         |
| LIVE           | Optimisation différentiable par image   | Calendrier [1, 2, 4, 8, 16, 1], 500 itérations par étape            | 32 chemins                            |
| Im2Vec         | Encodage et décodage de formes latentes | Checkpoint emoji, entrée interne $128 \times 128$                   | 4 chemins de 20 segments              |

### 3.6.2 Résultats quantitatifs communs

Le tableau 3.6 présente les résultats obtenus après relecture des SVG par le rasteriseur commun. SuperSVG obtient la meilleure moyenne pour les quatre métriques et se classe également premier sur chacune des quatre images prises séparément. Entre notre

approche et SuperSVG, la MSE passe de 0,00760 à 0,00151, soit une diminution absolue favorable de 0,00609, tandis que le PSNR gagne 6,55 dB. La SSIM passe de 0,648 à 0,831, soit une augmentation de 0,183, et la LPIPS diminue de 0,311 à 0,152, soit un écart favorable de 0,159.

TABLE 3.6 – Comparaison de la fidélité des approches sur les quatre images communes.

| Approche       | MSE ↓                                   | PSNR ↑                             | SSIM ↑                              | LPIPS ↓                             |
|----------------|-----------------------------------------|------------------------------------|-------------------------------------|-------------------------------------|
| Notre approche | $0,00760 \pm 0,00779$                   | $23,31 \pm 5,28$                   | $0,648 \pm 0,190$                   | $0,311 \pm 0,113$                   |
| SuperSVG       | <b><math>0,00151 \pm 0,00144</math></b> | <b><math>29,86 \pm 4,44</math></b> | <b><math>0,831 \pm 0,096</math></b> | <b><math>0,152 \pm 0,066</math></b> |
| LIVE           | $0,00658 \pm 0,00597$                   | $23,91 \pm 5,33$                   | $0,632 \pm 0,223$                   | $0,365 \pm 0,153$                   |
| Im2Vec         | $0,32384 \pm 0,13519$                   | $5,21 \pm 1,98$                    | $0,253 \pm 0,189$                   | $0,851 \pm 0,053$                   |

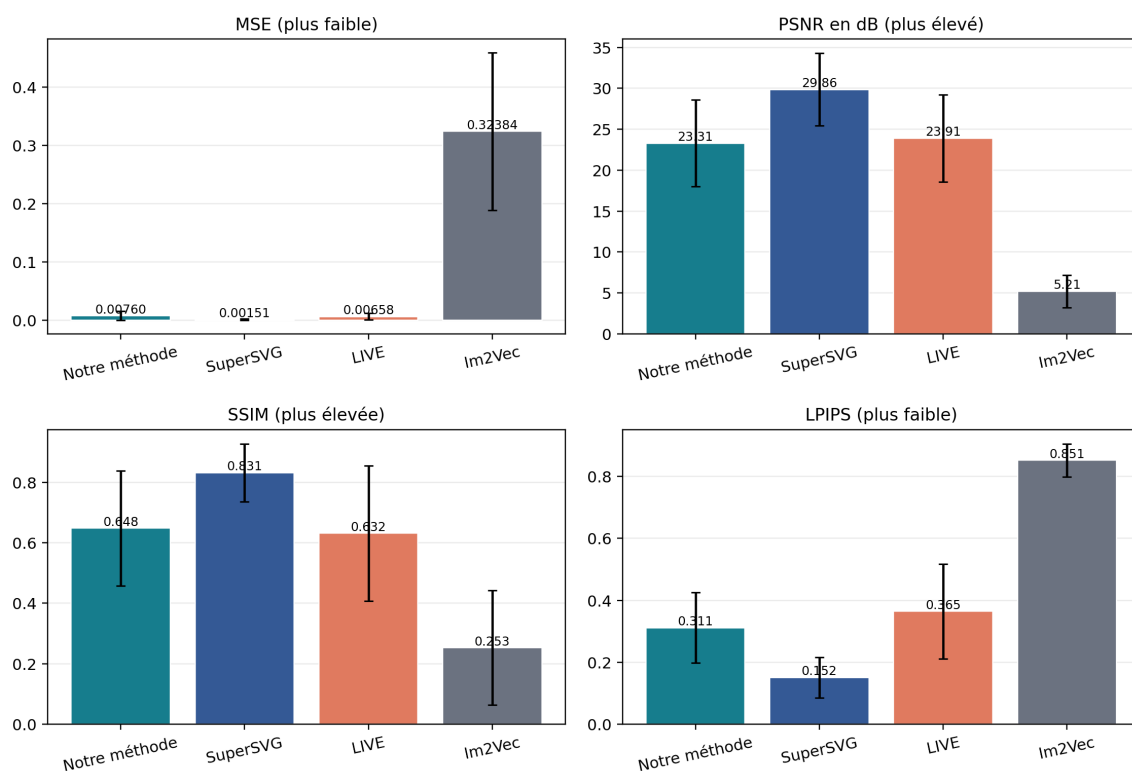


FIGURE 3.3 – Comparaison des quatre métriques de fidélité ; les barres d'erreur représentent l'écart-type entre les images.

Notre approche et LIVE présentent des moyennes plus proches, mais leur classement dépend de la métrique. Entre notre approche et LIVE, la MSE passe de 0,00760 à 0,00658, soit une diminution favorable de 0,00102, et le PSNR gagne 0,60 dB. Notre modèle conserve en revanche une SSIM de 0,648 contre 0,632, soit un avantage absolu de 0,016, ainsi qu'une LPIPS de 0,311 contre 0,365, inférieure de 0,054. Au niveau des images individuelles, notre approche dépasse LIVE sur trois images sur quatre pour la SSIM et la LPIPS, tandis que chaque approche obtient la meilleure MSE sur deux

images. Les mesures pixel par pixel favorisent donc légèrement LIVE en moyenne, alors que les mesures structurelle et perceptuelle favorisent notre modèle.

Le retrait d’Im2Vec doit être interprété au regard du décalage de domaine du checkpoint décrit précédemment. Les valeurs observées caractérisent ce modèle disponible hors de son domaine d’entraînement et ne permettent pas de conclure sur une architecture Im2Vec adaptée aux photographies.

La supériorité de SuperSVG est cohérente avec sa conception dédiée à la vectorisation par superpixels et avec son étape de raffinement, absente de notre version. Elle peut également dépendre du volume et de la stratégie d’entraînement de ses poids pré-entraînés. Comme ces facteurs ne sont pas isolés dans l’expérience, les résultats caractérisent les systèmes complets disponibles plutôt que l’apport causal d’un module particulier.

### 3.6.3 Temps et complexité vectorielle

Le tableau 3.7 montre que le classement de fidélité ne reflète pas directement le coût de production ou d’édition du SVG. Notre approche traite une image en 0,684 seconde en moyenne, contre 2,548 secondes pour SuperSVG. Elle est donc environ 3,72 fois plus rapide dans cet environnement, mais produit davantage de primitives : 1 472 chemins et 5 888 segments cubiques en moyenne, contre respectivement 1 000 et 4 000 pour SuperSVG.

Im2Vec est exclu de cette comparaison temporelle. Son checkpoint ne produit que quatre chemins et ses reconstructions des photographies du benchmark ne sont pas exploitables pour la tâche étudiée. Comparer directement sa vitesse à celle des trois approches qui reconstruisent effectivement le contenu reviendrait donc à opposer des charges vectorielles et des niveaux de fidélité sans rapport. Im2Vec reste néanmoins présent dans l’analyse quantitative et qualitative précédente afin de documenter ce comportement hors domaine.

TABLE 3.7 – Temps et complexité des SVG produits sur les quatre images communes.

| Approche       | Temps (s) ↓          | Chemins ↓        | Segments cubiques ↓ | SVG (Ko) ↓          |
|----------------|----------------------|------------------|---------------------|---------------------|
| Notre approche | $0,684 \pm 0,119$    | $1\,472 \pm 222$ | $5\,888 \pm 887$    | $662,10 \pm 102,30$ |
| SuperSVG       | $2,548 \pm 0,059$    | $1\,000 \pm 0$   | $4\,000 \pm 0$      | $544,04 \pm 1,50$   |
| LIVE           | $243,804 \pm 10,256$ | $32 \pm 0$       | $128 \pm 0$         | $21,01 \pm 0,02$    |

LIVE produit un document beaucoup plus compact, mais son optimisation progressive demande 243,804 secondes par image en moyenne, soit environ quatre minutes. Notre approche est ainsi approximativement 356 fois plus rapide. Dans les conditions du benchmark, cet écart met en évidence l’intérêt opérationnel d’une prédiction amortie :

le coût de l'apprentissage est payé en amont, puis chaque nouvelle image est traitée en une passe, tandis que LIVE recommence une optimisation de 3000 étapes pour chaque entrée [15]. En contrepartie, les 32 chemins de LIVE sont nettement plus faciles à parcourir et à modifier manuellement que les milliers de chemins produits par les approches à superpixels.

La figure 3.4 réunit ces compromis sur des axes logarithmiques. La position de SuperSVG traduit la meilleure fidélité avec une complexité élevée, celle de LIVE une forte compacité au prix du temps, et celle de notre approche un équilibre intermédiaire entre vitesse et qualité. La taille des marqueurs dépend de la taille moyenne du fichier SVG.

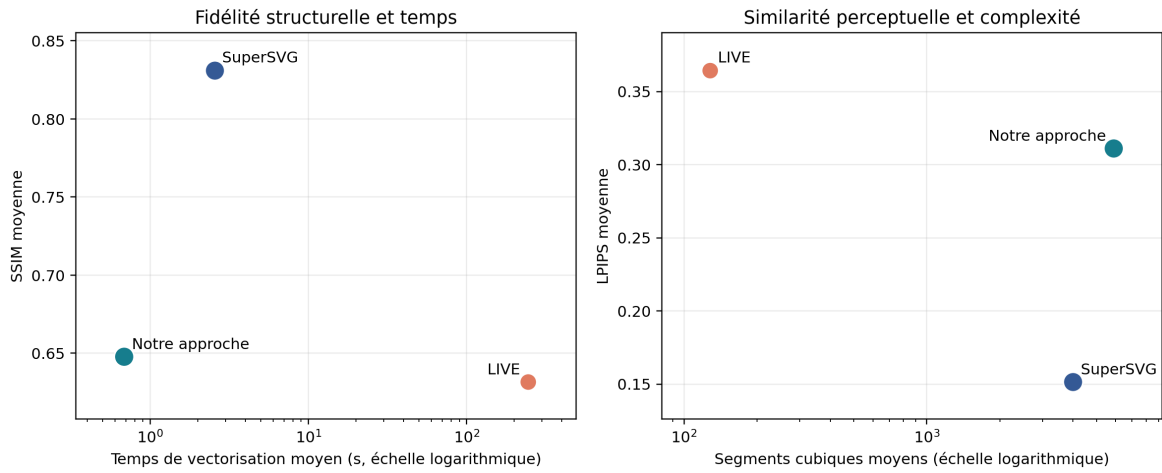


FIGURE 3.4 – Compromis entre fidélité, temps de vectorisation et complexité des trois approches retenues pour l'analyse temporelle.

### 3.6.4 Comparaison qualitative

La figure 3.5 présente les seize rendus utilisés pour les métriques. Sur la première image, SuperSVG restitue les deux animaux, le bâton et les transitions de couleur avec davantage de détails. Notre approche conserve la composition et les silhouettes principales, mais produit des aplats plus visibles. LIVE identifie les grandes régions, tout en simplifiant fortement les visages et l'arrière-plan en raison de son budget de 32 chemins.

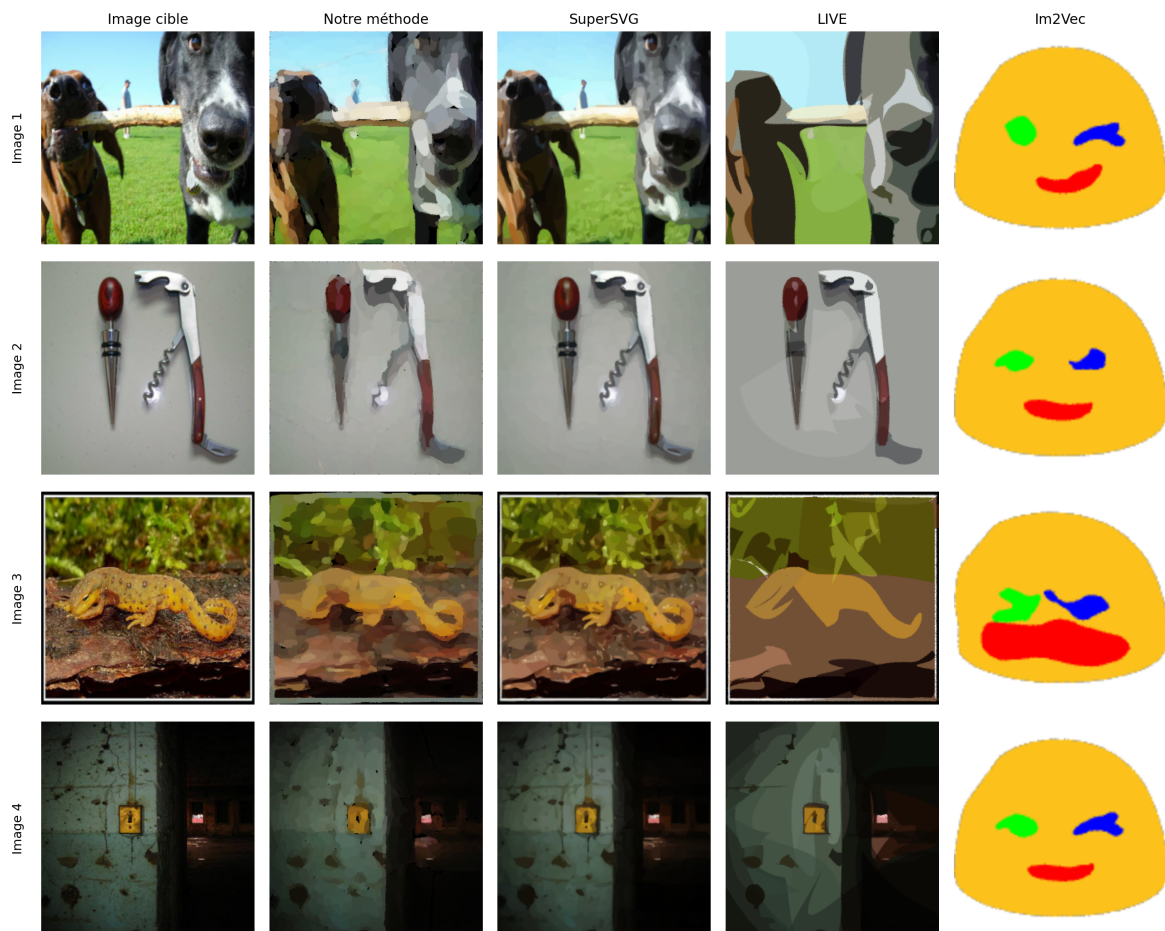


FIGURE 3.5 – Comparaison qualitative des SVG rasterisés par la procédure commune.

La deuxième ligne constitue le cas le plus favorable aux trois approches directement adaptées aux images naturelles. Le fond uniforme et les contours distincts permettent à SuperSVG de reproduire les ustensiles avec précision. Notre approche et LIVE conservent leurs silhouettes, mais les petites courbes métalliques et les ombres deviennent moins précises. Sur la troisième image, la texture végétale et les taches de la salamandre sont difficiles à représenter par des surfaces opaques. SuperSVG préserve le mieux ces variations, notre modèle en fournit une approximation colorimétrique plus grossière et LIVE réduit la scène à un nombre limité de grandes formes.

Dans la scène sombre de la quatrième ligne, SuperSVG et notre approche conservent la séparation entre le mur, la porte et les petits éléments lumineux. LIVE respecte la structure globale avec moins de détails, ce qui explique une SSIM plus faible malgré une MSE proche. Les quatre sorties Im2Vec prennent au contraire la forme d'un visage stylisé composé de zones jaune, verte, bleue et rouge, ce qui correspond au décalage de domaine déjà signalé.

### 3.6.5 Portée de la comparaison

Le benchmark commun améliore la comparabilité par rapport à l'utilisation de valeurs publiées : les cibles, la machine, le rasteriseur et les métriques sont identiques. Il reste néanmoins limité à quatre images et à des configurations de capacités différentes. Notre approche utilise un nombre variable de chemins, SuperSVG reçoit une cible de 1 000 chemins et LIVE en optimise 32. Im2Vec, qui n'en génère que quatre et ne reconstruit pas suffisamment les photographies, est exclu de l'analyse temporelle. Les résolutions internes, les domaines et les budgets d'entraînement diffèrent également, puisque nous évaluons les checkpoints disponibles sans réentraînement commun.

Ces résultats ne constituent donc pas un classement général des architectures. Ils décrivent le comportement des implémentations et des checkpoints disponibles sur un petit ensemble commun de photographies naturelles. Une comparaison contrôlée supplémentaire demanderait de réentraîner les modèles sur les mêmes données, d'imposer plusieurs budgets de segments identiques et d'élargir le jeu de test. Malgré cette réserve, l'expérience fournit une référence externe concrète qui complète l'étude interne de notre architecture.

## 3.7 Étude interne des configurations proposées

Après la comparaison externe, nous examinons plus précisément les deux configurations issues de notre entraînement. Cette étude met en relation leur fidélité de reconstruction avec le temps d'inférence et la complexité des documents SVG produits.

### 3.7.1 Fidélité de reconstruction

Le tableau 3.8 compare les deux configurations finales. Le modèle à 128 chemins obtient la meilleure moyenne pour chacune des quatre métriques. La MSE passe de 0,01615 à 0,00573, soit une diminution absolue favorable de 0,01042, et le PSNR gagne 5,00 dB. La SSIM passe de 0,438 à 0,645, soit une augmentation de 0,207, tandis que la LPIPS diminue de 0,492 à 0,290, soit un écart favorable de 0,202. L'amélioration concerne ainsi les mesures pixel par pixel, structurelles et perceptuelles.

TABLE 3.8 – Résultats moyens de fidélité sur les 12 images du jeu de test.

| Configuration | MSE ↓                    | PSNR ↑              | SSIM ↑               | LPIPS ↓              |
|---------------|--------------------------|---------------------|----------------------|----------------------|
| 32 chemins    | 0,01615 ± 0,01181        | 19,13 ± 3,54        | 0,438 ± 0,152        | 0,492 ± 0,058        |
| 128 chemins   | <b>0,00573 ± 0,00533</b> | <b>24,13 ± 4,05</b> | <b>0,645 ± 0,135</b> | <b>0,290 ± 0,075</b> |

L'amélioration est observée sur les 12 images pour la MSE, la SSIM et la LPIPS, et

non uniquement sur la moyenne. La figure 3.6 présente les moyennes et les écarts-types. La dispersion reste importante pour la MSE et le PSNR, car la difficulté varie fortement entre une scène texturée contenant plusieurs objets et une image sombre dominée par de grandes zones uniformes.

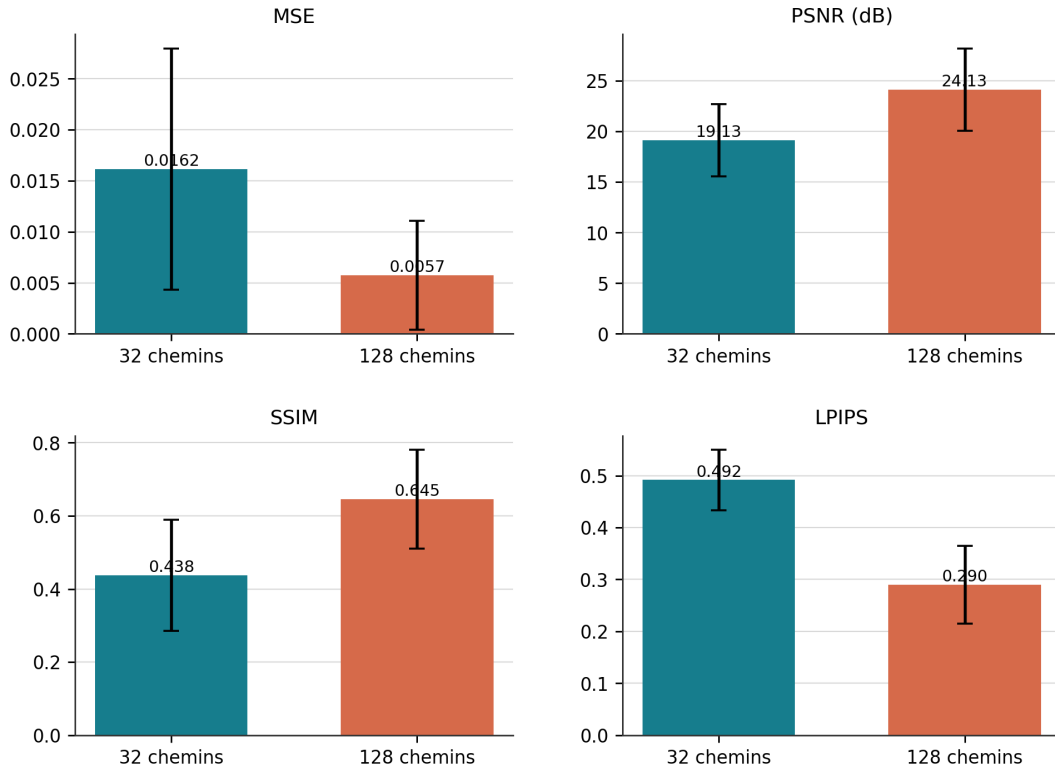


FIGURE 3.6 – Comparaison quantitative des configurations finales ; les barres d’erreur représentent l’écart-type.

La meilleure valeur de SSIM du modèle à 128 chemins, égale à 0,848, est obtenue sur l’image contenant les deux ustensiles placés sur un fond relativement uniforme. Le PSNR maximal atteint 29,83 dB sur une scène sombre dont de larges régions sont peu texturées. À l’inverse, l’image de la salamandre entourée d’une végétation détaillée obtient une SSIM de 0,465, malgré l’augmentation du nombre de chemins. Ces écarts montrent que les formes principales et les aplats de couleur sont plus faciles à approximer que les petites variations répétées de texture.

Compte tenu des budgets précisés dans le tableau 3.2, ces écarts portent sur les deux configurations complètes et ne permettent pas d’isoler l’effet du seul nombre de primitives.

### 3.7.2 Temps d’inférence et complexité du SVG

L’augmentation de la fidélité s’accompagne d’un coût vectoriel. Le tableau 3.9 détaille les temps moyens. La préparation SLIC et le passage dans le réseau restent

presque constants. En revanche, le rendu passe de 0,155 à 0,475 seconde et l’export de 0,040 à 0,141 seconde. Le temps total mesuré est ainsi multiplié par 2,67.

TABLE 3.9 – Coût moyen de l’inférence et complexité des SVG générés.

| Mesure                          | 32 chemins          | 128 chemins          |
|---------------------------------|---------------------|----------------------|
| Préparation des régions (s)     | $0,0366 \pm 0,0044$ | $0,0349 \pm 0,0039$  |
| Passage dans le réseau (s)      | $0,0208 \pm 0,0020$ | $0,0206 \pm 0,0019$  |
| Rendu DiffVG (s)                | $0,1547 \pm 0,0312$ | $0,4752 \pm 0,0933$  |
| Export SVG (s)                  | $0,0398 \pm 0,0303$ | $0,1408 \pm 0,0633$  |
| Temps total mesuré (s)          | $0,2520 \pm 0,0483$ | $0,6715 \pm 0,1361$  |
| Mémoire GPU supplémentaire (Mo) | $80,1 \pm 7,5$      | $80,9 \pm 6,3$       |
| Nombre de chemins               | $354,7 \pm 76,5$    | $1\,418,7 \pm 306,1$ |
| Taille du SVG (Ko)              | $158,9 \pm 34,2$    | $633,4 \pm 136,9$    |

Le nombre de chemins et la taille du fichier sont presque exactement multipliés par quatre, car les deux configurations utilisent les mêmes régions et sérialisent respectivement 32 et 128 primitives pour chacune. La mémoire GPU supplémentaire reste proche de 81 Mo. Ce résultat s’explique par le fait que le backbone domine le nombre de paramètres et que les scènes DiffVG sont rendues par image. Le principal coût de la configuration à 128 chemins est donc temporel et documentaire, plutôt que lié à une forte augmentation de la mémoire du réseau.

La figure 3.7 visualise ce compromis. DiffVG et l’export représentent la majorité du temps, surtout pour le modèle à 128 chemins. Une réduction de la complexité après la prédiction, par exemple par suppression des chemins redondants ou fusion de formes proches, pourrait donc diminuer à la fois le temps de sérialisation, la taille du fichier et la difficulté d’édition.

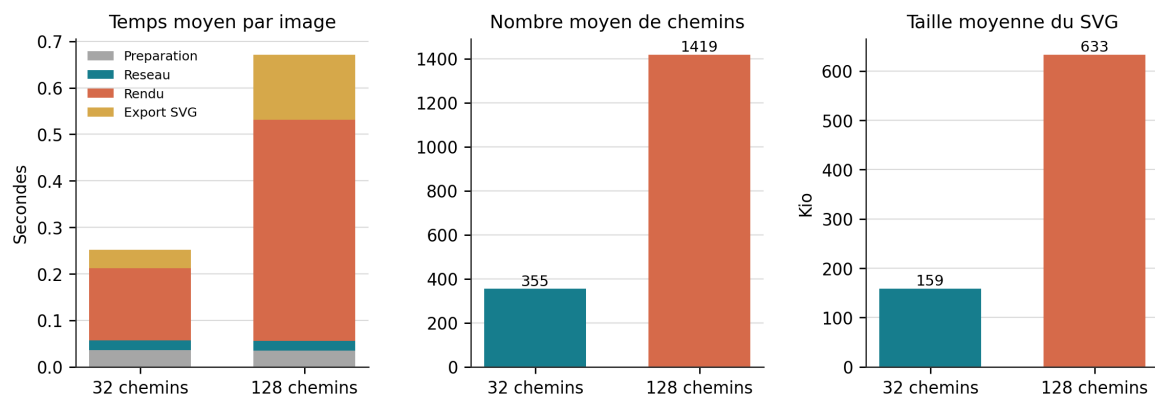


FIGURE 3.7 – Répartition du temps moyen et évolution de la complexité vectorielle.

### 3.8 Analyse qualitative de l'approche proposée

La figure 3.8 présente trois exemples choisis pour illustrer des comportements différents. Elle compare l'image matricielle, le rendu du modèle à 32 chemins et celui du modèle à 128 chemins. Les valeurs indiquées sous les reconstructions correspondent aux métriques propres à chaque image.

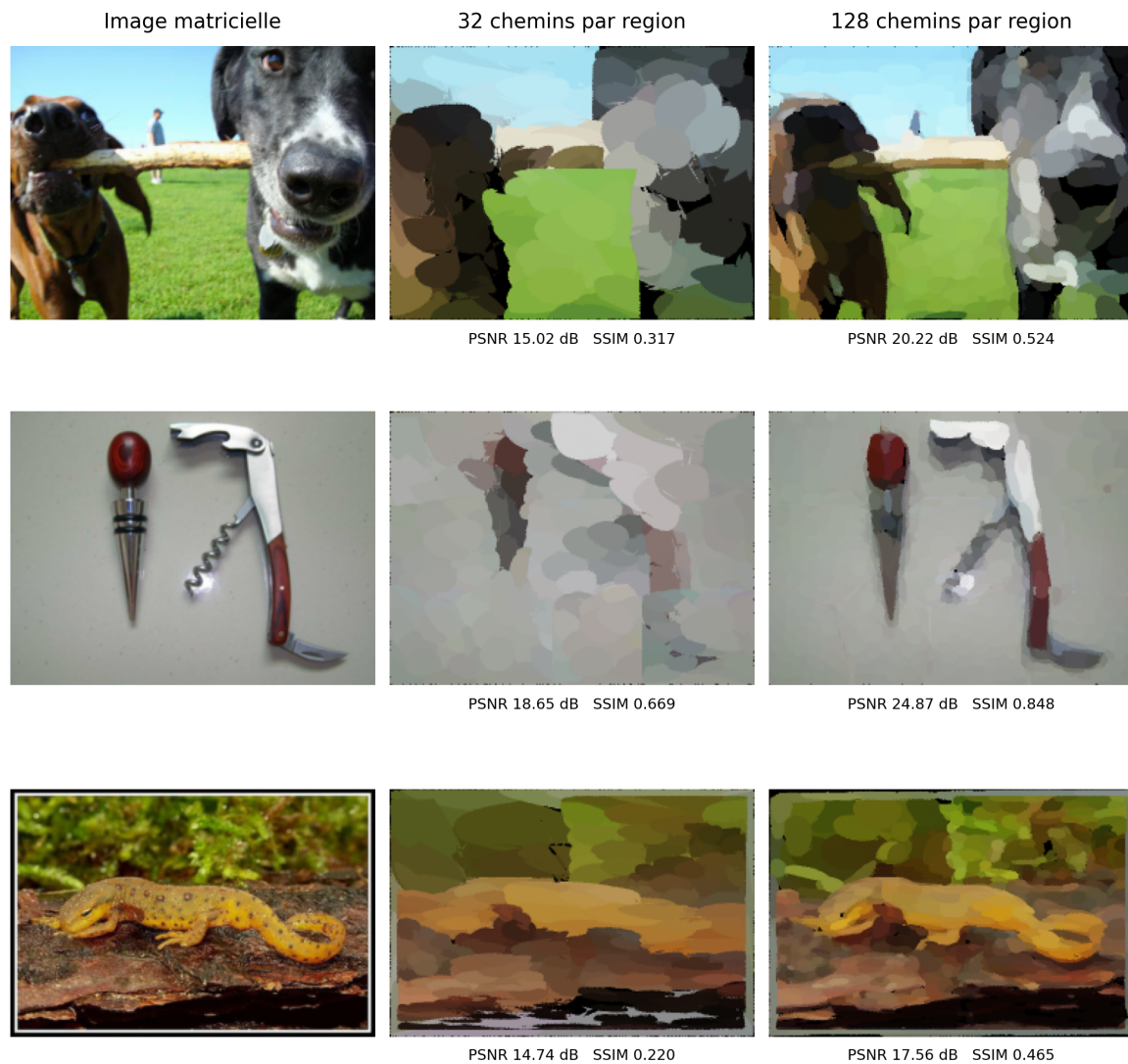


FIGURE 3.8 – Comparaison qualitative sur une scène complexe, un objet isolé et une image fortement texturée.

Dans la première ligne, la configuration à 32 chemins reconstruit les principales masses de couleur, notamment le ciel, l'herbe et les deux animaux, mais plusieurs détails du visage sont confondus. Avec 128 chemins, les yeux, les museaux, le bâton et les transitions de couleur deviennent plus reconnaissables. Le PSNR passe de 15,02 à 20,22 dB, soit un gain de 5,20 dB, et la SSIM de 0,317 à 0,524, soit une augmentation absolue de 0,207. La reconstruction reste cependant stylisée et composée d'aplats superposés

plutôt que de textures proches de la photographie.

La deuxième ligne montre un cas plus favorable. Le fond est presque uniforme et les objets possèdent des silhouettes distinctes. La configuration à 32 chemins restitue mal les deux ustensiles, car une partie importante des primitives est utilisée pour approximer le fond et les ombres. Le modèle à 128 chemins sépare clairement les deux objets, retrouve leurs couleurs dominantes et conserve la forme générale du tire-bouchon. La SSIM atteint alors 0,848.

La troisième ligne contient une salamandre sur un arrière-plan riche en mousse et en bois. Le modèle à 128 chemins améliore la silhouette et la répartition des couleurs, mais ne reproduit pas les petites taches ni les variations fines de la végétation. Le résultat à 32 chemins réduit l'animal à une forme allongée et plusieurs zones du fond fusionnent. Cet exemple confirme que l'augmentation du nombre de primitives aide à représenter les détails, mais ne résout pas entièrement la perte d'information provoquée par la segmentation locale et par la représentation en surfaces opaques.

Plusieurs artefacts sont communs aux résultats. De minces zones noires peuvent apparaître lorsque les formes ne couvrent pas complètement une frontière, tandis que des superpositions produisent parfois une transition abrupte entre régions. Les chemins étant fermés, sans contour et sans transparence, le modèle doit représenter toutes les variations par des aplats colorés. Enfin, les frontières de SLIC restent visibles dans certaines reconstructions, car les régions sont prédites indépendamment puis concaténées sans étape globale de raccordement.

Sur le plan de l'éditabilité, tous les fichiers sont de véritables SVG composés de chemins de Bézier et peuvent être ouverts par DiffVG. Néanmoins, un document contenant en moyenne plus de 1 400 chemins devient difficile à modifier manuellement. La configuration à 32 chemins est plus compacte, mais sa simplification visuelle est nettement plus forte. La qualité d'une approche de vectorisation ne doit donc pas être jugée uniquement par la similarité matricielle : le nombre et l'organisation des primitives restent déterminants.

### 3.9 Discussion et limites

Les résultats montrent que le réseau produit en une seule passe des primitives conduisant à des SVG valides, tandis que les métriques évoluent favorablement au cours de l'apprentissage. Dans le benchmark externe, SuperSVG offre la meilleure fidélité, notre approche occupe une position intermédiaire entre vitesse et qualité, et LIVE privilégie la compacité au prix d'une optimisation beaucoup plus longue. L'étude interne met également en évidence un compromis : la configuration à 128 chemins améliore la reconstruction, mais augmente le temps de rendu et la complexité documentaire.

Ces constats restent circonscrits aux conditions définies dans la section 3.6. Le

nombre réduit d’images, l’hétérogénéité des configurations disponibles, l’exécution temporelle unique par image et l’absence de vérité terrain SVG empêchent d’en déduire un classement général. Les mesures caractérisent la fidélité matricielle, le coût et la structure des fichiers produits dans notre environnement ; elles n’évaluent pas directement leur proximité avec un document conçu par un graphiste.

Sur le plan architectural, le nombre de chemins fixe par superpixel entraîne des primitives redondantes dans les régions simples, tandis que le traitement indépendant des régions peut créer des discontinuités. Les surfaces opaques sans traits ni dégradés limitent en outre la représentation des textures fines. Une capacité adaptative, une simplification des chemins après prédiction et un raffinement global *coarse-to-fine* constituent ainsi des prolongements cohérents. Des expériences contrôlées avec des budgets d’entraînement identiques seraient nécessaires pour isoler l’effet du nombre de chemins, de l’ajustement du backbone DINOv3 et des paramètres SLIC.

### 3.10 Conclusion

Dans ce chapitre, nous avons vérifié la chaîne de rendu différentiable et la validité des SVG, puis évalué notre architecture selon deux niveaux complémentaires. La comparaison externe place SuperSVG en tête pour la fidélité, tandis que notre approche réduit le temps de vectorisation au prix d’un plus grand nombre de primitives ; LIVE produit des documents plus compacts mais nécessite une optimisation sensiblement plus longue.

L’étude interne montre que la configuration à 128 chemins améliore les quatre métriques par rapport à celle à 32 chemins, avec un coût supérieur en temps de rendu et en complexité du SVG. Ces observations soutiennent la faisabilité de la prédiction directe de primitives à partir du backbone DINOv3, tout en orientant la suite du travail vers un raffinement visuel et une capacité vectorielle plus adaptative.

# Conclusion Générale

Ce projet de fin d'études a porté sur la vectorisation automatique d'images matricielles par apprentissage profond. Nous avons conçu une chaîne qui décompose l'image avec SLIC, extrait les caractéristiques de chaque région avec DINOv3 et prédit, au moyen de mécanismes d'attention, des chemins fermés composés de courbes de Bézier cubiques. Grâce à DiffVG, l'entraînement repose uniquement sur une comparaison matricielle et ne nécessite pas de SVG de référence.

Les vérifications techniques ont confirmé la transmission des gradients et la validité des SVG générés. L'étude interne montre que la configuration à 128 chemins obtient les résultats les plus favorables dans les conditions évaluées, sans permettre d'isoler l'effet du nombre de chemins en raison de budgets d'entraînement différents. Dans le benchmark externe, SuperSVG conserve la meilleure fidélité, notre approche privilégie la rapidité au prix de documents plus complexes, et LIVE produit des SVG plus compacts mais nécessite une optimisation plus longue. Ces observations restent limitées par la taille du jeu de test et par l'hétérogénéité des configurations comparées.

Les principales limites concernent le nombre fixe de chemins, le traitement indépendant des superpixels et la représentation fondée uniquement sur des formes fermées et opaques. Les travaux futurs pourront donc porter sur une capacité adaptative, la simplification des primitives et un raffinement global de type *coarse-to-fine*. Un benchmark plus étendu et des entraînements réalisés dans des conditions identiques permettront également d'évaluer plus précisément chaque composante. Malgré ces limites, le système développé constitue une chaîne fonctionnelle de conversion d'une image matricielle vers un document SVG éditabile.

# Bibliographie

- [1] Rafael C. GONZALEZ et Richard E. WOODS. *Digital Image Processing*. 4th. Pearson, 2018. ISBN : 978-1-292-22304-9.
- [2] James D. FOLEY et al. *Computer Graphics : Principles and Practice*. 2nd. Addison-Wesley Professional, 1996, p. 8.
- [3] Radhakrishna ACHANTA et al. “SLIC Superpixels Compared to State-of-the-Art Superpixel Methods”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.11 (2012), p. 2274-2282. DOI : 10.1109/TPAMI.2012.120.
- [4] Tzu-Mao LI et al. “Differentiable Vector Graphics Rasterization for Editing and Learning”. In : *ACM Transactions on Graphics* 39.6 (2020), 193 :1-193 :15. DOI : 10.1145/3414685.3417871.
- [5] Oriane SIMÉONI et al. “DINOv3”. In : (2025). DOI : 10.48550/arXiv.2508.10104. arXiv : 2508.10104 [cs.CV].
- [6] W3C. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. World Wide Web Consortium, 2011. URL : <https://www.w3.org/TR/SVG11/>.
- [7] Aurélien GÉRON. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd. O’Reilly Media, 2022. ISBN : 978-1-098-12597-4.
- [8] Tom M. MITCHELL. *Machine Learning*. McGraw-Hill, 1997, p. 2.
- [9] Christopher M. BISHOP. *Pattern Recognition and Machine Learning*. Springer, 2006, p. 3.
- [10] Longlong JING et Yingli TIAN. “Self-Supervised Visual Feature Learning with Deep Neural Networks : A Survey”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11 (2021), p. 4037-4058. DOI : 10.1109/TPAMI.2020.2992393.
- [11] Yann LECUN et al. “Backpropagation Applied to Handwritten Zip Code Recognition”. In : *Neural Computation* 1.4 (1989), p. 541-551. DOI : 10.1162/neco.1989.1.4.541.
- [12] Ashish VASWANI et al. “Attention Is All You Need”. In : *Advances in Neural Information Processing Systems*. T. 30. Curran Associates, Inc., 2017.
- [13] Alexey DOSOVITSKIY et al. “An Image Is Worth 16x16 Words : Transformers for Image Recognition at Scale”. In : *International Conference on Learning Representations*. 2021. URL : <https://arxiv.org/abs/2010.11929>.

- [14] Teng HU et al. “SuperSVG : Superpixel-based Scalable Vector Graphics Synthesis”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, p. 24892-24901.
- [15] Xu MA et al. “Towards Layer-wise Image Vectorization”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, p. 16314-16323. DOI : 10.1109/CVPR52688.2022.01583.
- [16] Zhou WANG et al. “Image Quality Assessment : From Error Visibility to Structural Similarity”. In : *IEEE Transactions on Image Processing* 13.4 (2004), p. 600-612. DOI : 10.1109/TIP.2003.819861.
- [17] Richard ZHANG et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, p. 586-595. DOI : 10.1109/CVPR.2018.00068.
- [18] Maria DZIUBA et al. *Image Vectorization : A Review*. 2023. arXiv : 2306.06441 [cs.CV].
- [19] Peter SELINGER. *Potrace : A Polygon-Based Tracing Algorithm*. Technical report. Sept. 2003. URL : <https://potrace.sourceforge.net/potrace.pdf>.
- [20] Pradyumna REDDY et al. “Im2Vec : Synthesizing Vector Graphics without Vector Supervision”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, p. 7342-7351.
- [21] Jia DENG et al. “ImageNet : A Large-Scale Hierarchical Image Database”. In : *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2009, p. 248-255. DOI : 10.1109/CVPR.2009.5206848.